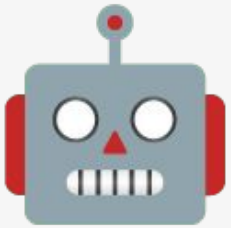
아직도 직접 구현중?

LLM 직접 구현할 시간에
난 깐x치킨 간다

박경선 | enfj.dev@gmail.com

LinkedIn : @gyeongsun-park
Github : @gngsn

Contents



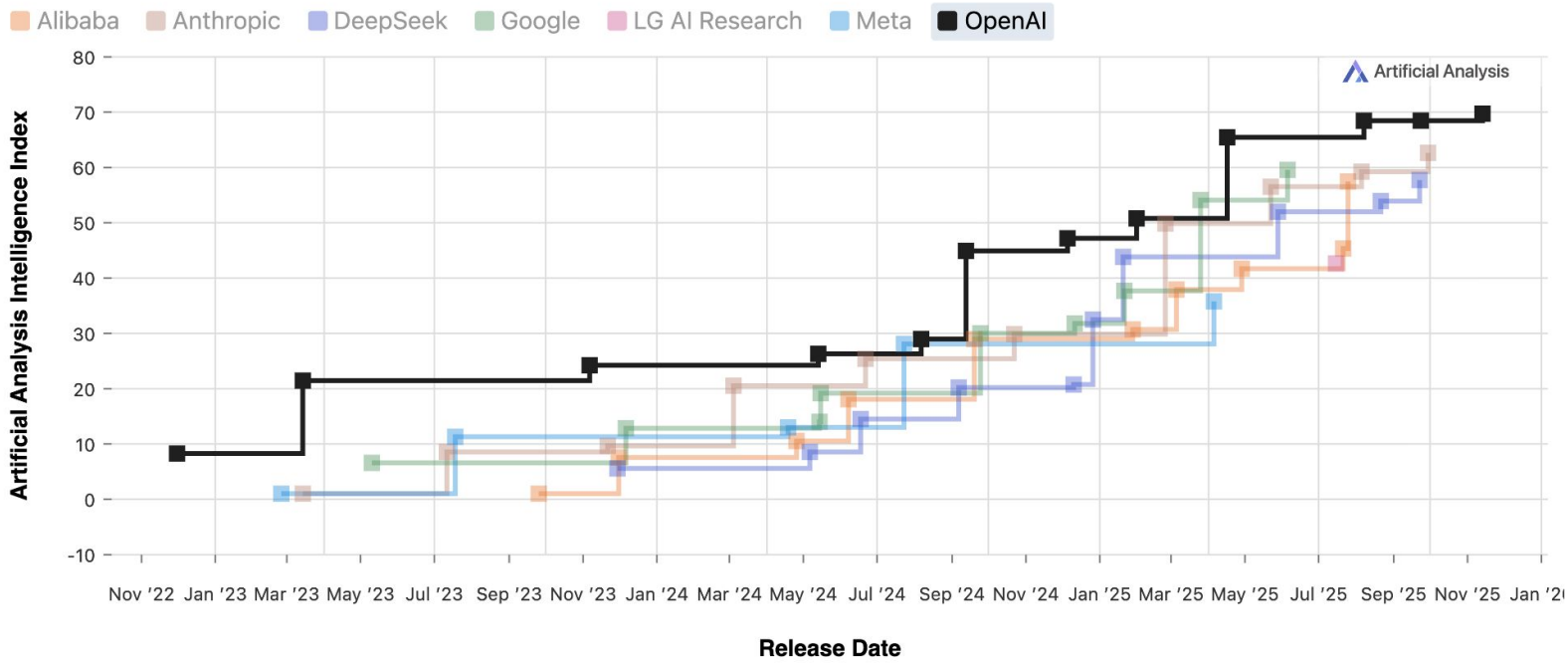
AI

LLM

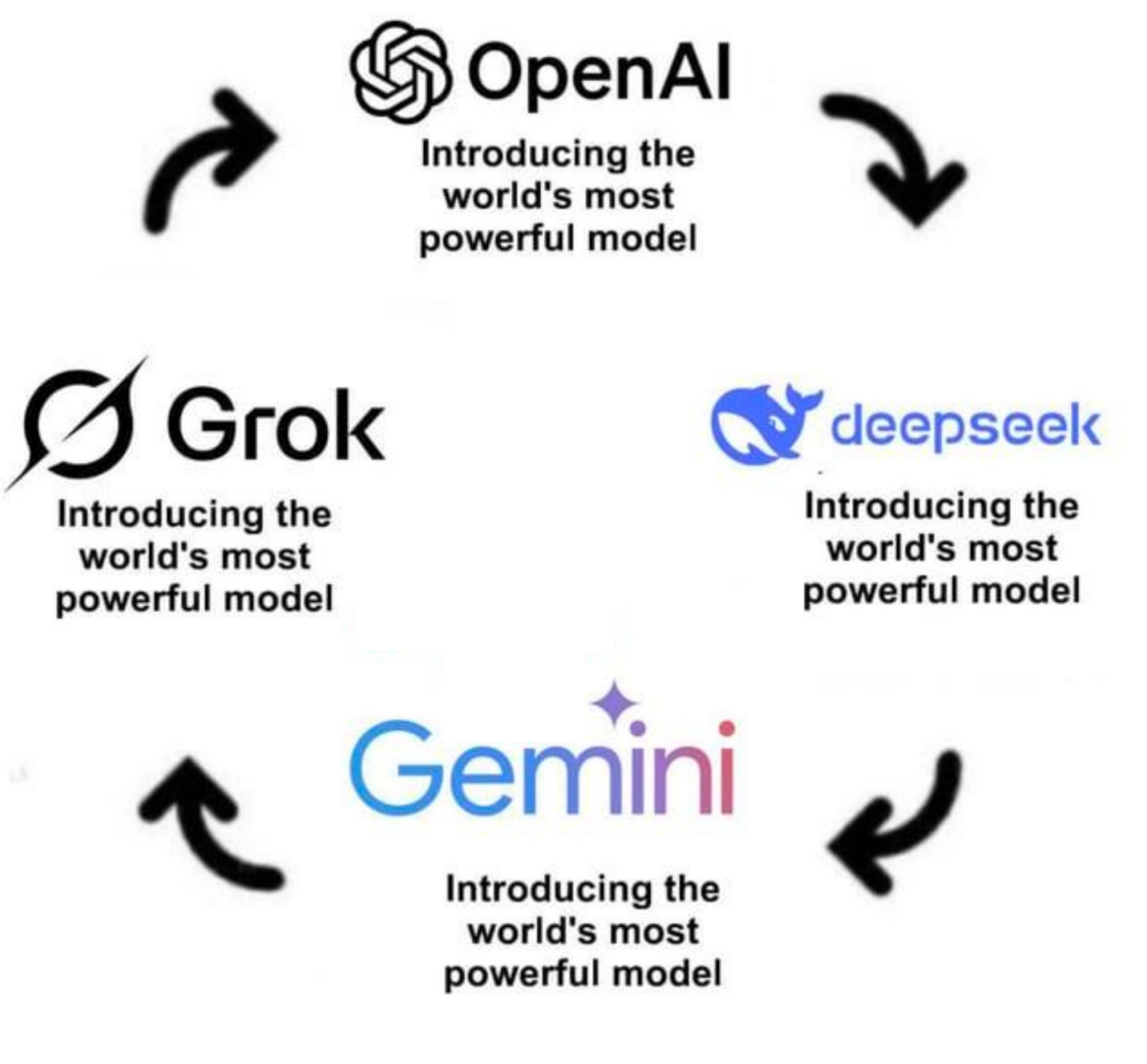

UPDATE
ChatGPT
UPDATE
Llama
UPDATE
Gemini

Frontier Language Model Intelligence, Over Time

Artificial Analysis Intelligence Index v3.0 incorporates 10 evaluations: MMLU-Pro, GPQA Diamond, Humanity's Last Exam, LiveCodeBench, SciCode, AIME 2025, IFBench, AA-LCR, Terminal-Bench Hard, τ^2-Bench Telecom



https://artificialanalysis.ai


OpenAI
Introducing the world's most powerful model
Grok
Introducing the world's most powerful model
deepseek
Introducing the world's most powerful model
Gemini
Introducing the world's most powerful model

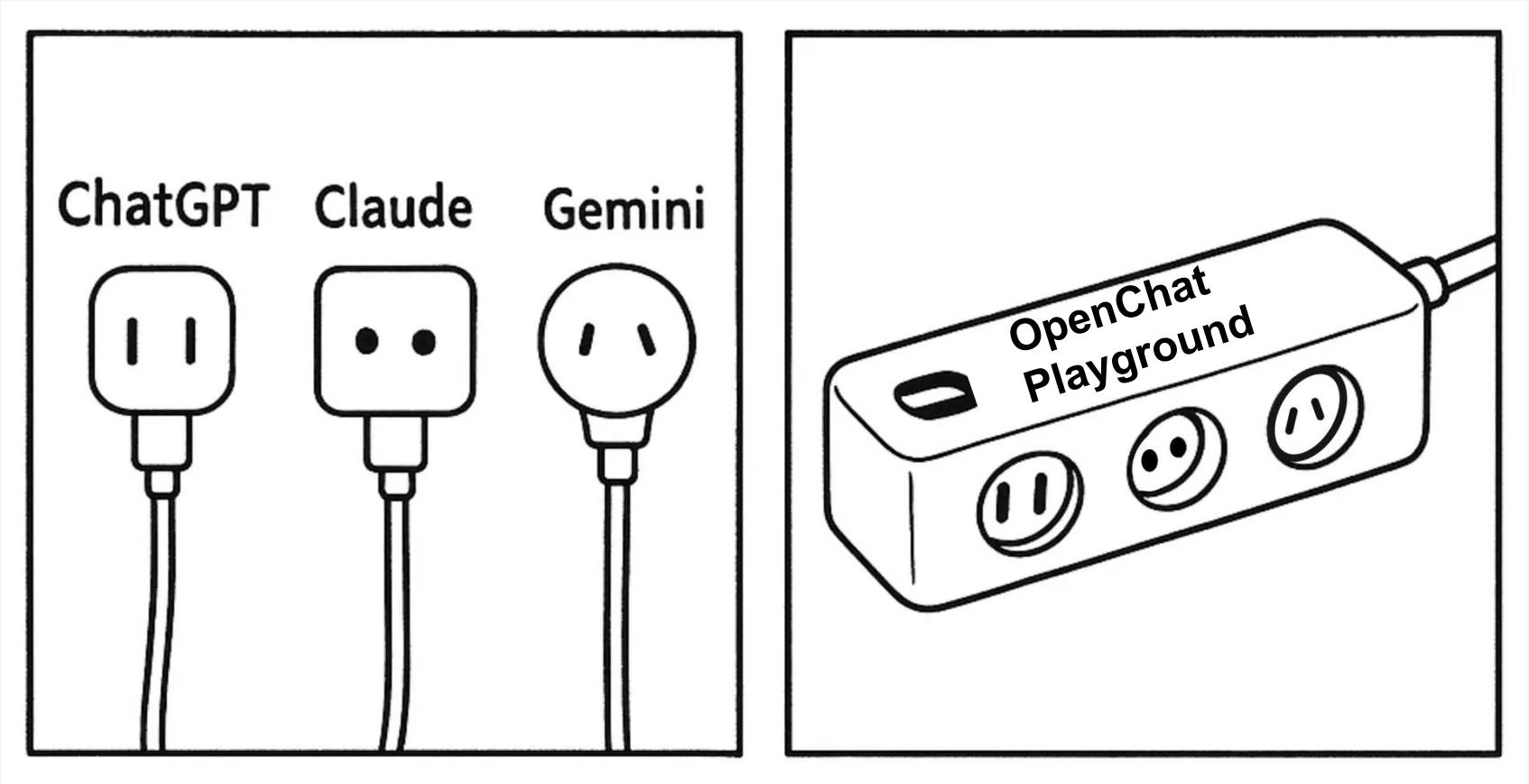
📚 학습 시간

⚙️ 구현 시간

🧪 테스트

🆕 배포

독자적인 구현 및 실행 방식을 가진 모델


ChatGPT
Claude
Gemini
OpenChat
Playground

OpenChat Playground

https://bit.ly/openchat-playground

https://bit.ly/openchat-playground

aliencube/open-chat-playgro

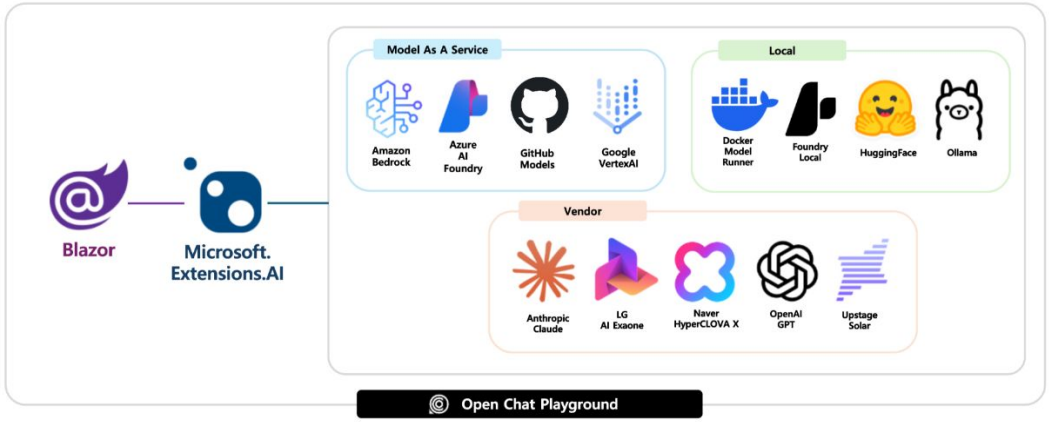
github.com/aliencube/open-chat-playground

Relaunch to update

README License

Open Chat Playground

Open Chat Playground (OCP) is a web UI that is able to connect virtually any LLM from any platform.

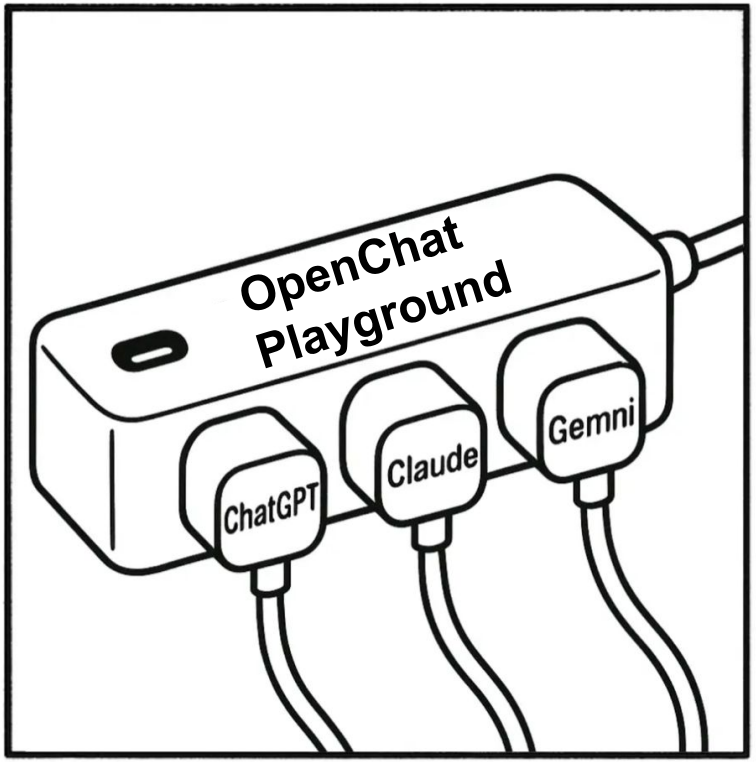


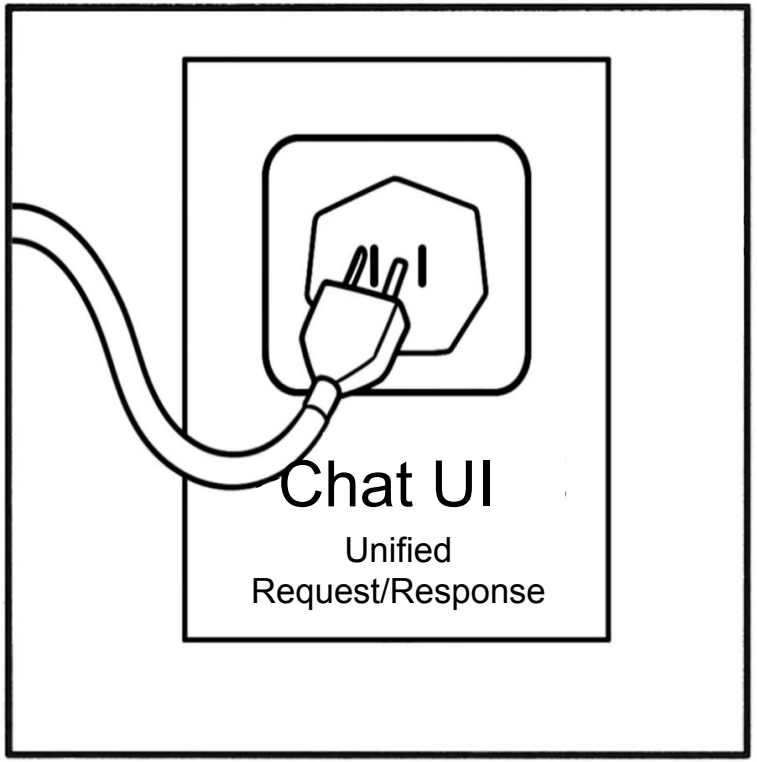
Supported platforms

- [x] Amazon Bedrock
- [x] Azure AI Foundry

Languages

C# 86.2% JavaScript 8.2%
Bicep 2.6% CSS 1.4%
HTML 1.3% Shell 0.2%
Dockerfile 0.1%


OpenChat
Playground
ChatGPT
Claude
Gemni


Chat UI
Unified
Request/Response

Porter AI

LLM Providers

Open Chat Playground

Model As A Service


Amazon Bedrock


Azure AI Foundry


Github Models


Google VertexAI

LLM Providers

Open Chat Playground

Model As A Service

Amazon Bedrock

Azure AI Foundry

Github Models

Google VertexAI

Local

Docker Model Runner

Foundry Local

HuggingFace

Ollama

LLM Providers

Open Chat Playground

Model As A Service

Amazon Bedrock

Azure AI Foundry

Github Models

Google VertexAI

Local

Docker Model Runner

Foundry Local

HuggingFace

Ollama

Vendor

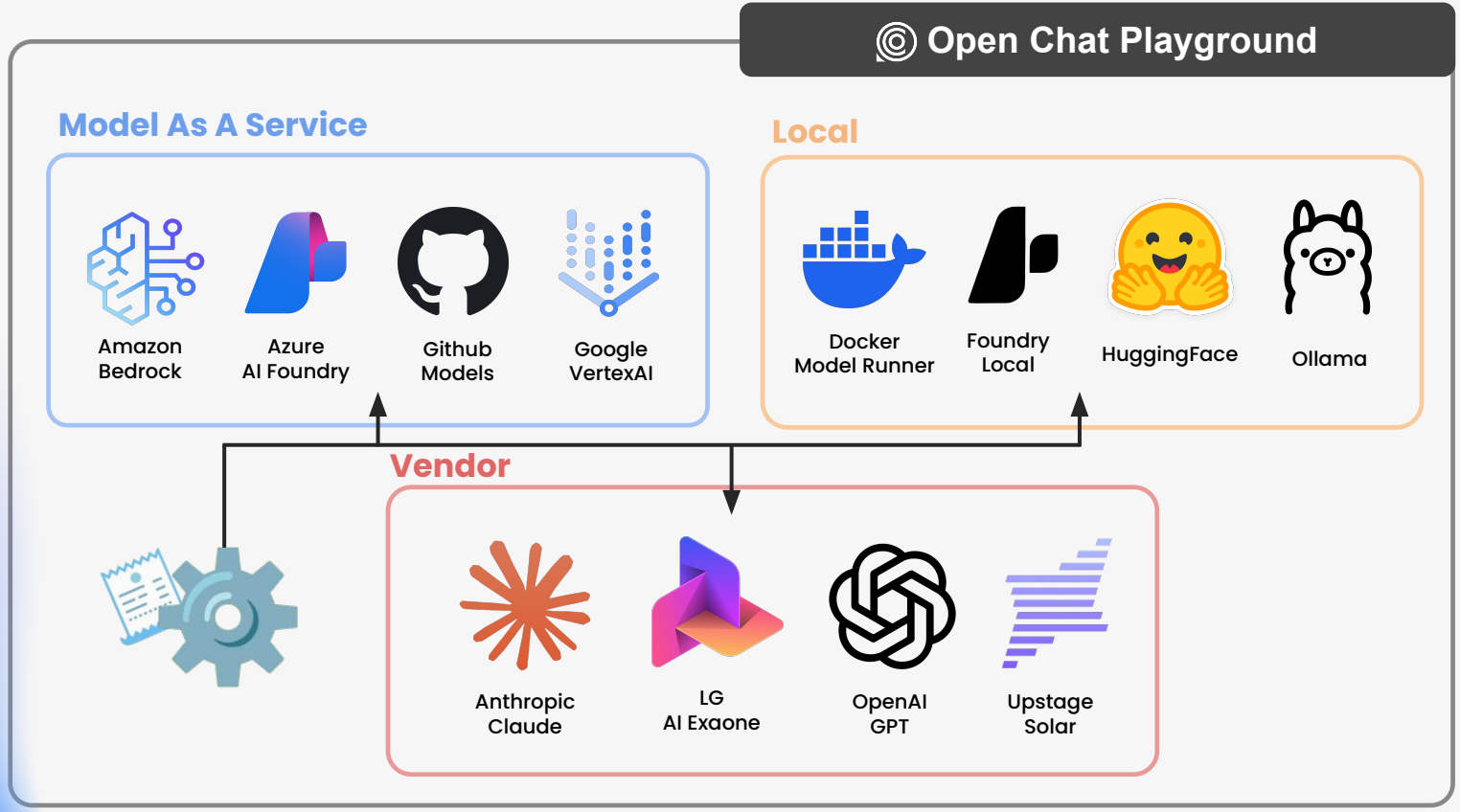
Anthropic Claude

LG AI Exaone

OpenAI GPT

Upstage Solar

LLM Providers



쉬운 LLM 교체 —

✅ 빠른 **최신 모델** 적용

✅ **벤더 종속성 방지** Vendor Lock-in 회피

✅ 자유로운 **실험 · A/B 테스트**

✅ 비용·용도에 **최적화된 모델 선택**

쉬운 LLM 교체 —

- ✅ 빠른 **최신 모델** 적용
- ✅ **벤더 종속성 방지** Vendor Lock-in 회피
- ✅ 자유로운 **실험 · A/B 테스트**
- ✅ 비용·용도에 **최적화된 모델 선택**

쉬운 LLM 교체 —

✅ 빠른 **최신 모델** 적용

✅ **벤더 종속성 방지** Vendor Lock-in 회피

✅ 자유로운 **실험 · A/B 테스트**

✅ 비용·용도에 **최적화된 모델 선택**

쉬운 LLM 교체 —

✅ 빠른 **최신 모델** 적용

✅ **벤더 종속성 방지** Vendor Lock-in 회피

✅ 자유로운 **실험 · A/B 테스트**

✅ 비용·용도에 **최적화된 모델 선택**

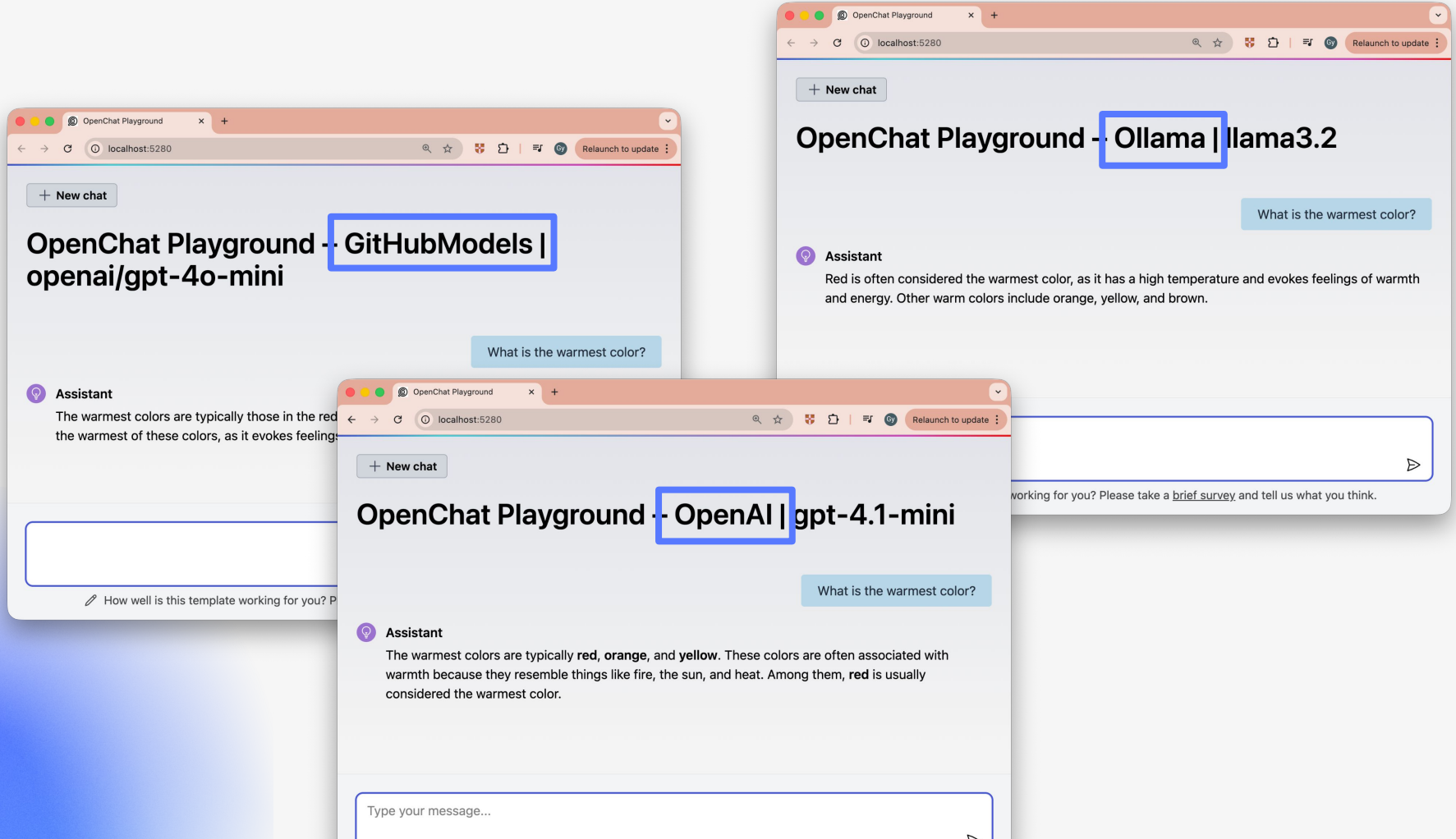
쉬운 LLM 교체 —

✅ 빠른 **최신 모델** 적용

✅ **벤더 종속성 방지** Vendor Lock-in 회피

✅ 자유로운 **실험 · A/B 테스트**

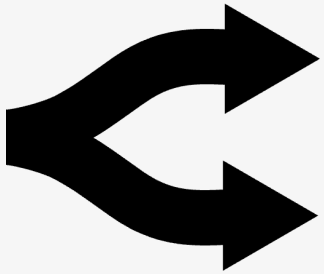
✅ 비용·용도에 **최적화된 모델 선택**


OpenChat Playground
localhost:5280
Relaunch to update
New chat
OpenChat Playground -- GitHubModels | openai/gpt-4o-mini
What is the warmest color?
Assistant
The warmest colors are typically those in the red
the warmest of these colors, as it evokes feeling
How well is this template working for you? P
OpenChat Playground
localhost:5280
Relaunch to update
New chat
OpenChat Playground -- OpenAI | gpt-4.1-mini
What is the warmest color?
Assistant
The warmest colors are typically red, orange, and yellow. These colors are often associated with warmth because they resemble things like fire, the sun, and heat. Among them, red is usually considered the warmest color.
Type your message...
OpenChat Playground
localhost:5280
Relaunch to update
New chat
OpenChat Playground -- Ollama | llama3.2
What is the warmest color?
Assistant
Red is often considered the warmest color, as it has a high temperature and evokes feelings of warmth and energy. Other warm colors include orange, yellow, and brown.
working for you? Please take a brief survey and tell us what you think.

기존에 없었을까?

기존에 없었을까?

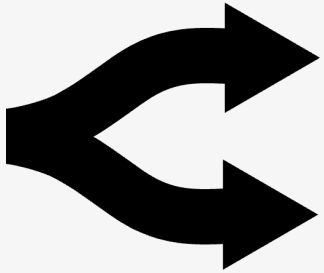
Open WebUI



Open Router

기존에 없었을까?

Open WebUI



Open Router

✔️ 제한적인 모델 제공

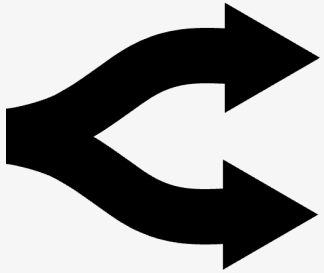
✔️ **API 지원 X**

기존에 없었을까?

Open WebUI

✔️ 제한적인 모델 제공

✔️ **API 지원 X**



Open Router

✔️ 유료 서비스

OpenChat Playground

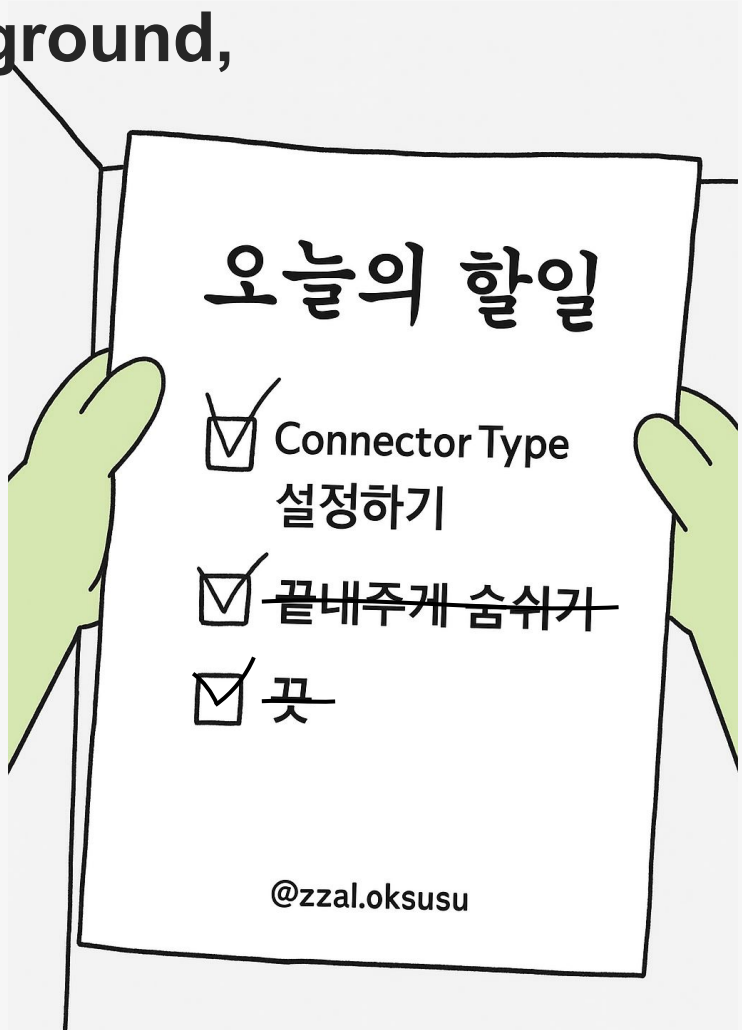
✔️ Local 부터 Vendor, MaaS 모델 제공자 지원

✔️ API 지원

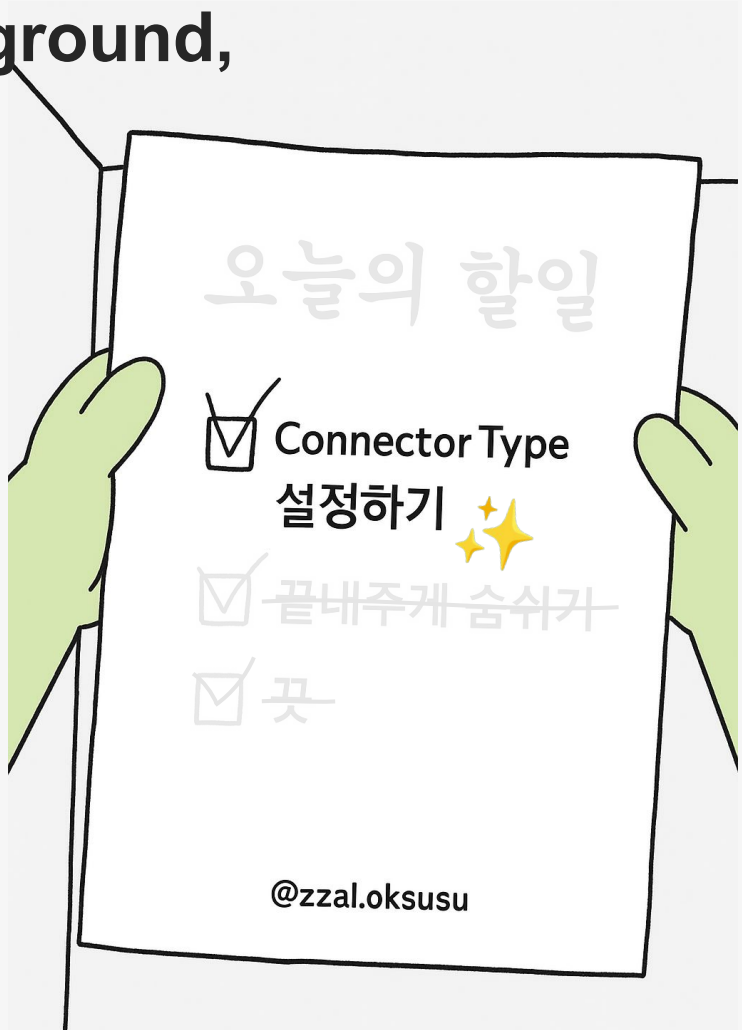
✔️ OpenSource 프로젝트

How?

OpenChat Playground,
실행하려면?



OpenChat Playground,
실행하려면?



ConnectorType 설정

- 📌 appsettings.json
- 📌 Environment Variables
- 📌 Command-Line Arguments

1️⃣ appsettings.json 파일 설정

src/OpenChat.PlaygroundApp/appsettings.json

```
{
    ...
    "ConnectorType": "GitHubModels",
    ...
}
```

```
dotnet run
```

appsettings.json

src > OpenChat.PlaygroundApp > appsettings.json > ...

```
{
  "ConnectorType": "GitHubModels",

  "AmazonBedrock": {
    "AccessKeyId": "{{AMAZON_BEDROCK_ACCESS_KEY_ID}}",
    "SecretAccessKey": "{{AMAZON_BEDROCK_SECRET_ACCESS_KEY}}",
    "Region": "{{AMAZON_BEDROCK_REGION}}",
    "ModelId": "amazon.nova-micro-v1:0"
  },

  "AzureAIFoundry": {
    "Endpoint": "{{AZURE_AI_FOUNDRY_ENDPOINT}}",
    "ApiKey": "{{AZURE_AI_FOUNDRY_API_KEY}}",
    "DeploymentName": "gpt-4o-mini"
  },

  "GitHubModels": {
    "Endpoint": "https://models.github.ai/inference",
    "Token": "{{GITHUB_PAT}}",
    "Model": "openai/gpt-4o-mini"
  },

  "GoogleVertexAI": {
    "ApiKey": "{{GOOGLE_VERTEX_AI_API_KEY}}",
    "Model": "gemini-2.5-flash-lite"
```

📑 appsettings.json

: LLM 모델 별 기본 설정

✅ **비민감 정보 설정:**

- 기본 값이 미리 지정되어 있음
- 명령 시 인자로 전달

✅ **민감 정보 설정:**

- .NET Secret Manager tool (`dotnet user-secret`)
- 명령 시 인자로 전달

Run on local machine

1. Make sure you are at the repository root.

```
cd $REPOSITORY_ROOT
```

2. Add GitHub Personal Access Token (PAT) for GitHub Models connection. Make sure you should replace `{{GH_PAT}}` with your GitHub PAT.

```
# bash/zsh
dotnet user-secrets --project $REPOSITORY_ROOT/src/OpenChat.PlaygroundApp \
    set GitHubModels:Token "{{GH_PAT}}"
```

```
# PowerShell
dotnet user-secrets --project $REPOSITORY_ROOT/src/OpenChat.PlaygroundApp `
    set GitHubModels:Token "{{GH_PAT}}"
```

> For more details about GitHub PAT, refer to the doc, Managing your personal access tokens.

3. Run the app. The default model OCP uses is GPT-4o mini.

Code File Edit Selection View Go Run Terminal Window Help

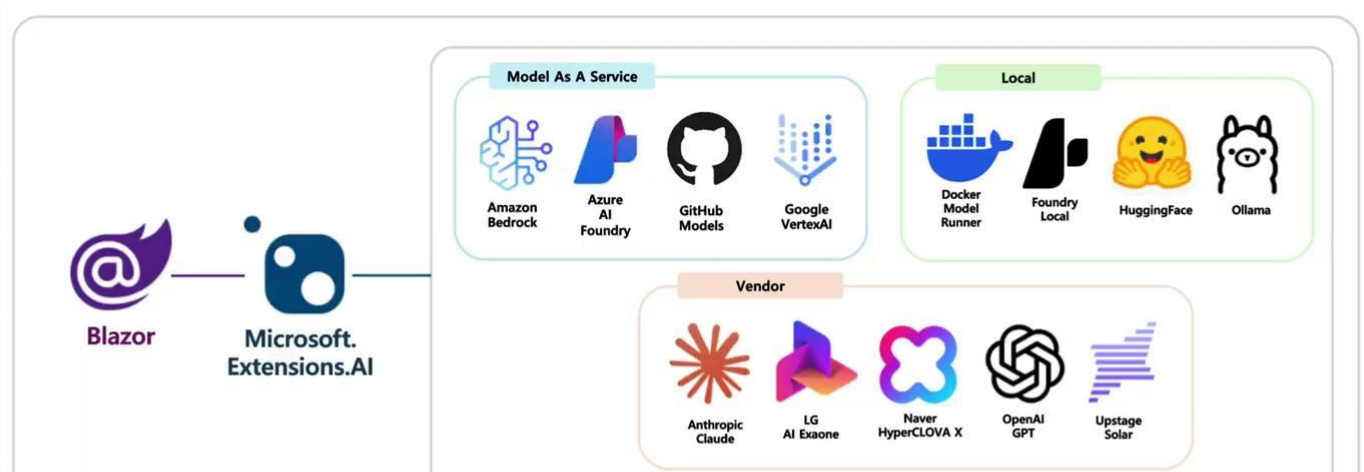
Mon Nov 24 12:17 PM

open-chat-playground

Preview README.md

Open Chat Playground

Open Chat Playground (OCP) is a web UI that is able to connect virtually any LLM from any platform.



PROBLEMS 8 OUTPUT DEBUG CONSOLE TERMINAL PORTS zsh - OpenChat.PlaygroundApp

~/g/0/open-chat-playground/s/OpenChat.PlaygroundApp on main ↑2 base at 12:17:28

main 0↓ 2↑ ⊗ 0 ⚠ 8 Projects: 4 Debug Any CPU Solution file opened: OpenChatPlayground.sln Git Graph

2 Environment Variables 설정

```
# Linux / MacOS
export CONNECTOR_TYPE=OpenAI

# Windows
set CONNECTOR_TYPE=OpenAI
```

```
dotnet run
```

Code File Edit Selection View Go Run Terminal Window Help

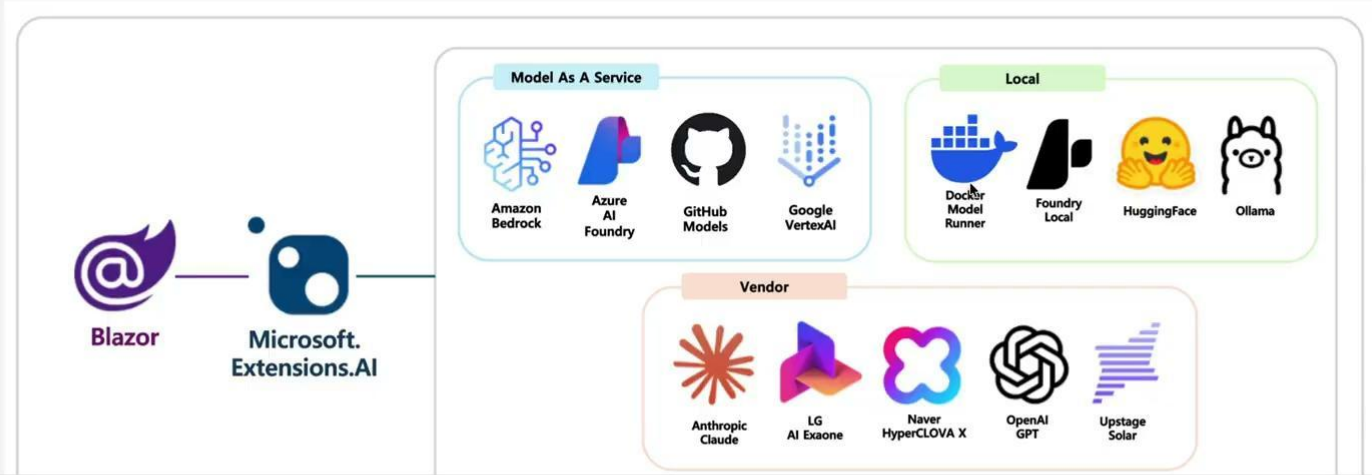
Mon Nov 24 12:21 PM

open-chat-playground

Preview README.md

Open Chat Playground

Open Chat Playground (OCP) is a web UI that is able to connect virtually any LLM from any platform.



PROBLEMS 8 OUTPUT DEBUG CONSOLE TERMINAL PORTS

~/g/0/open-chat-playground/s/OpenChat.PlaygroundApp on main ↑2

zsh...

zsh...

main 0↓ 2↑ 0 8 Projects: 4 Debug Any CPU Solution file opened: OpenChatPlayground.sln Git Graph

3 Command-Line Arguments 설정

```
dotnet run -- --connector-type OpenAI
```

Code File Edit Selection View Go Run Terminal Window Help

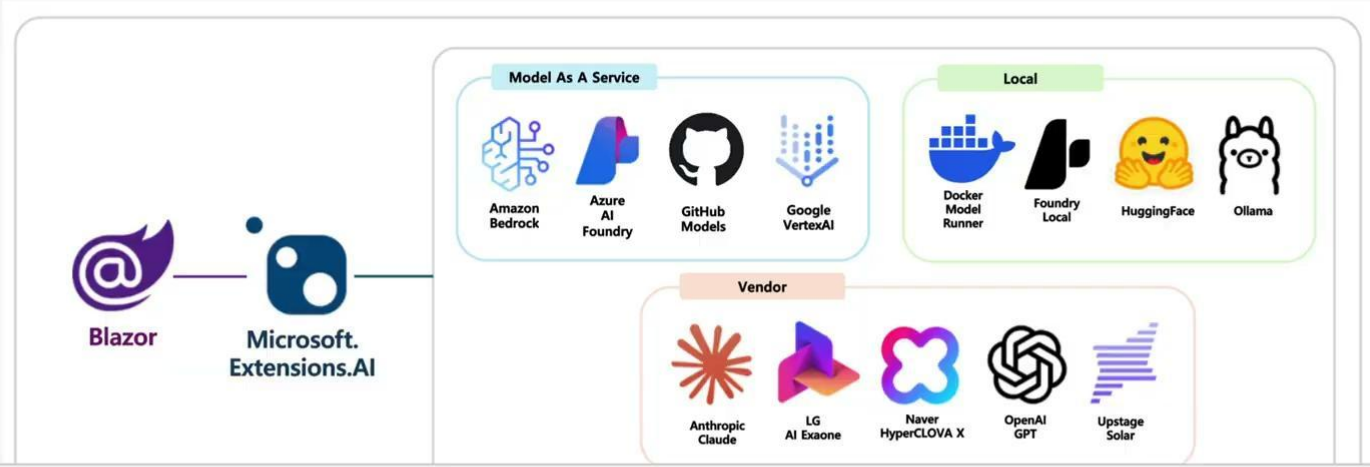
Mon Nov 24 12:23 PM

open-chat-playground

Preview README.md

Open Chat Playground

Open Chat Playground (OCP) is a web UI that is able to connect virtually any LLM from any platform.



PROBLEMS 8 OUTPUT DEBUG CONSOLE TERMINAL PORTS

~/g/0/open-chat-playground/s/OpenChat.PlaygroundApp on main ↑2

olla...

zsh...

main 0↓ 2↑ 0 8 Projects: 4 Debug Any CPU Solution file opened: OpenChatPlayground.sln Git Graph

ConnectorType 적용 순서

📌 appsettings.json

📌 Environment Variables

📌 Command-Line Arguments

ConnectorType 적용 순서

📌 appsettings.json 🥉

📌 Environment Variables 🥈

📌 Command-Line Arguments 🥇

ConnectorType 적용 순서

📌 appsettings.json

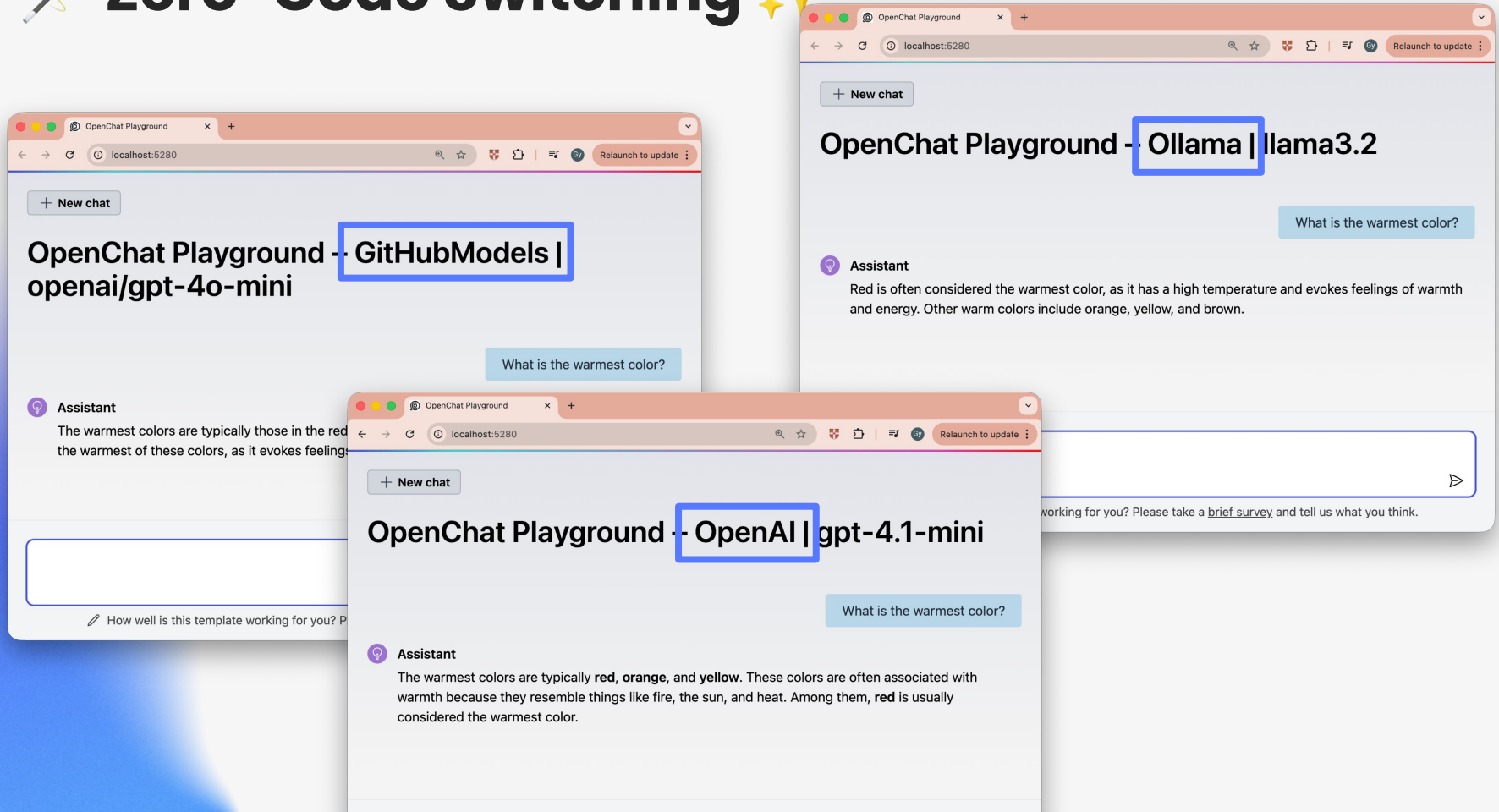
↓

📌 Environment Variables

↓

📌 Command-Line Arguments

Zero-Code Switching



How?

독자적인 구현 및 실행 방식을 가진
각 모델 별 SDKs

- Amazon Bedrock SDK
- Ollama SDK
- Azure AI Foundry SDK
- Anthropic SDK
- Google Vertex AI SDK
- OpenAI SDK

독자적인 구현 및 실행 방식을 가진
각 모델 별 SDKs

문제점

1. 서로 다른 Request/Response 구조

2. 규격화의 부재

3. SDK 업데이트마다 유지보수가 발생

독자적인 구현 및 실행 방식을 가진
각 모델 별 SDKs

문제점

1. 서로 다른 Request/Response 구조

호출 함수 형식, JSON 형식, API 규격 등 상이

2. 규격화의 부재

3. SDK 업데이트마다 유지보수가 발생

독자적인 구현 및 실행 방식을 가진
각 모델 별 SDKs

문제점

1. 서로 다른 Request/Response 구조

2. 규격화의 부재

인증 방식, 모델 초기화 요구사항, 이미지·음성 입력 등

3. SDK 업데이트마다 유지보수가 발생

독자적인 구현 및 실행 방식을 가진
각 모델 별 SDKs

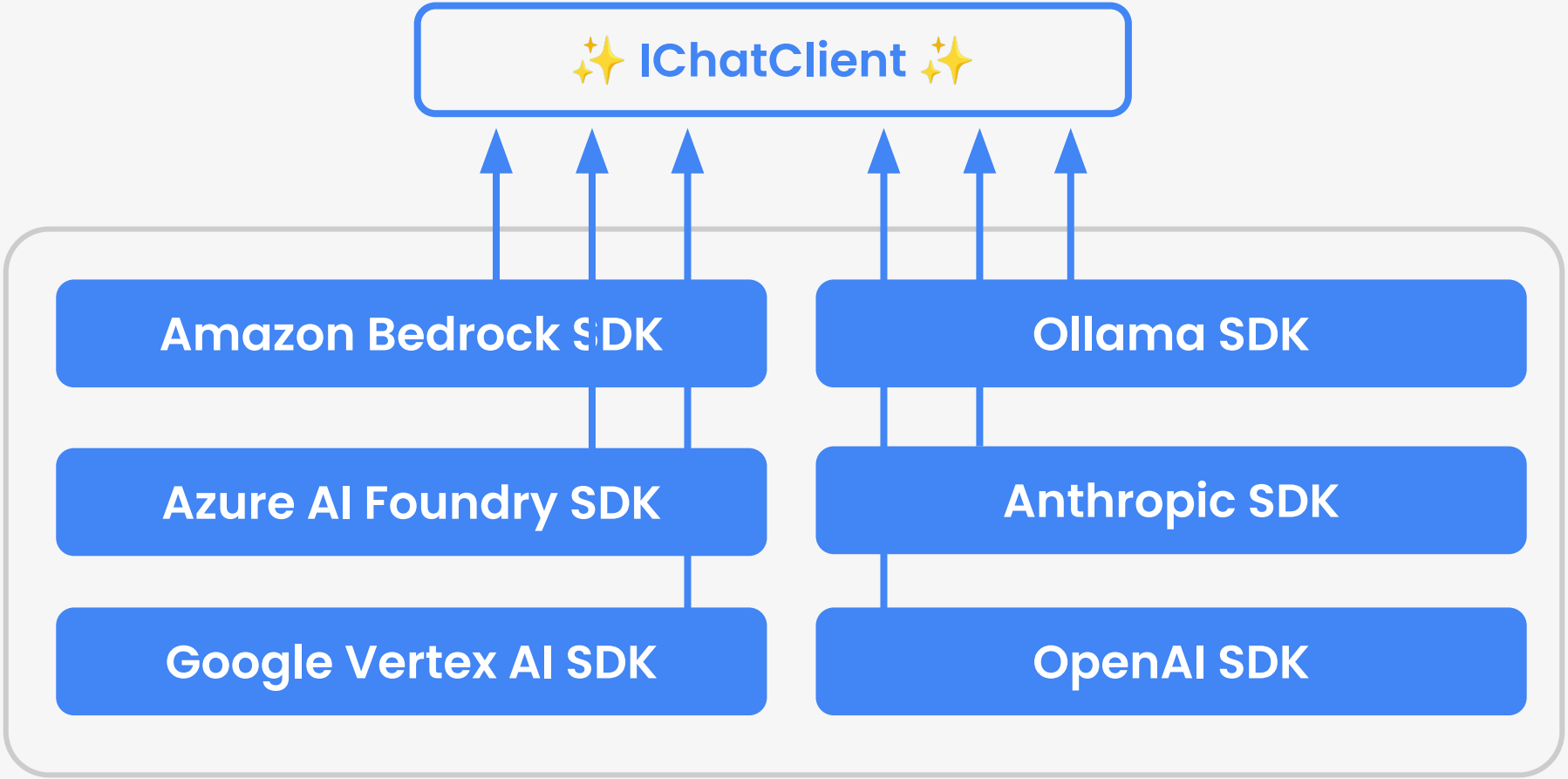
문제점

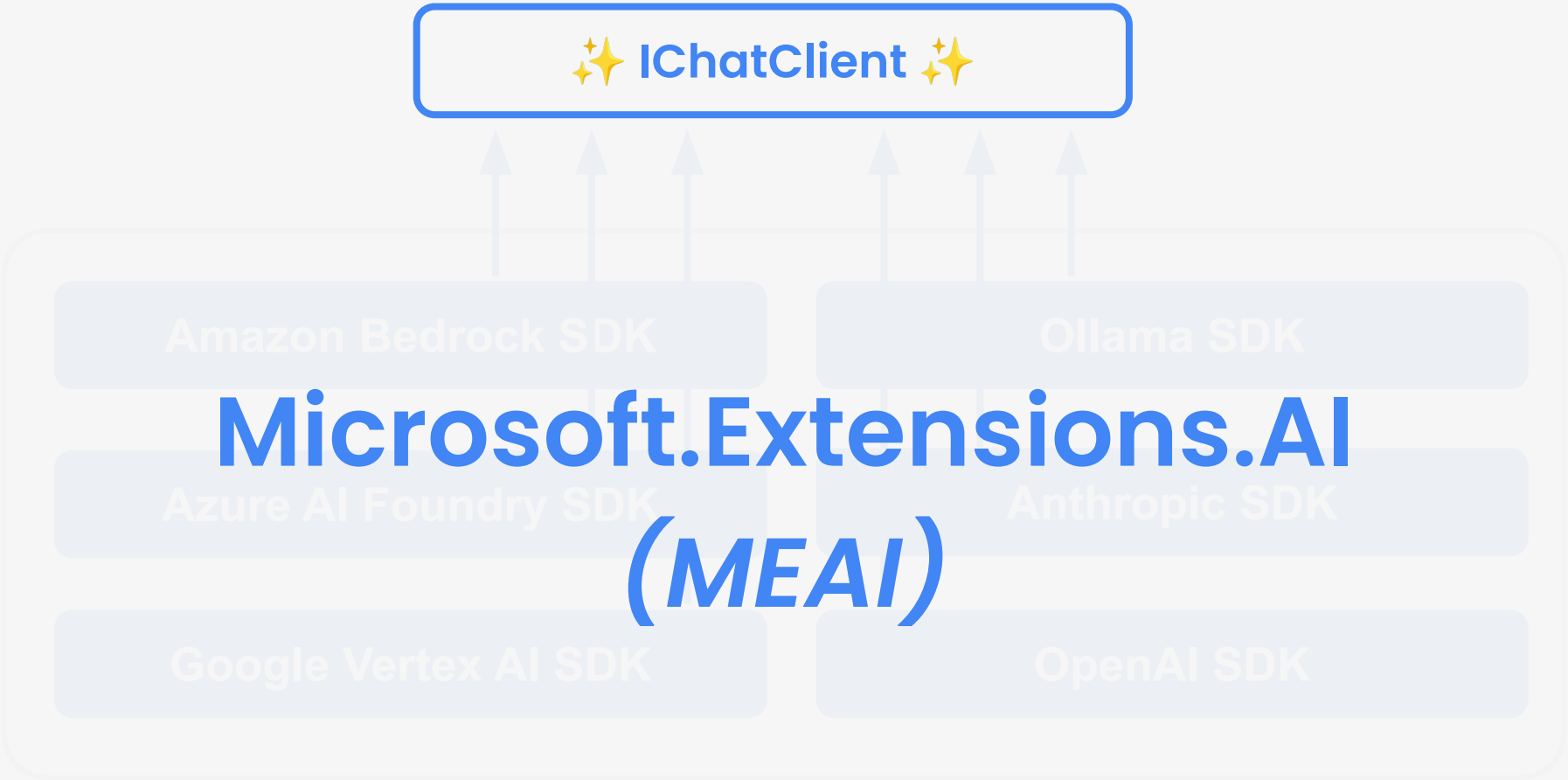
1. 서로 다른 Request/Response 구조

2. 규격화의 부재

3. SDK 업데이트마다 유지보수가 발생

각 업데이트마다 모두 개별적으로 대응


✨ IChatClient ✨
Amazon Bedrock SDK
Ollama SDK
Azure AI Foundry SDK
Anthropic SDK
Google Vertex AI SDK
OpenAI SDK

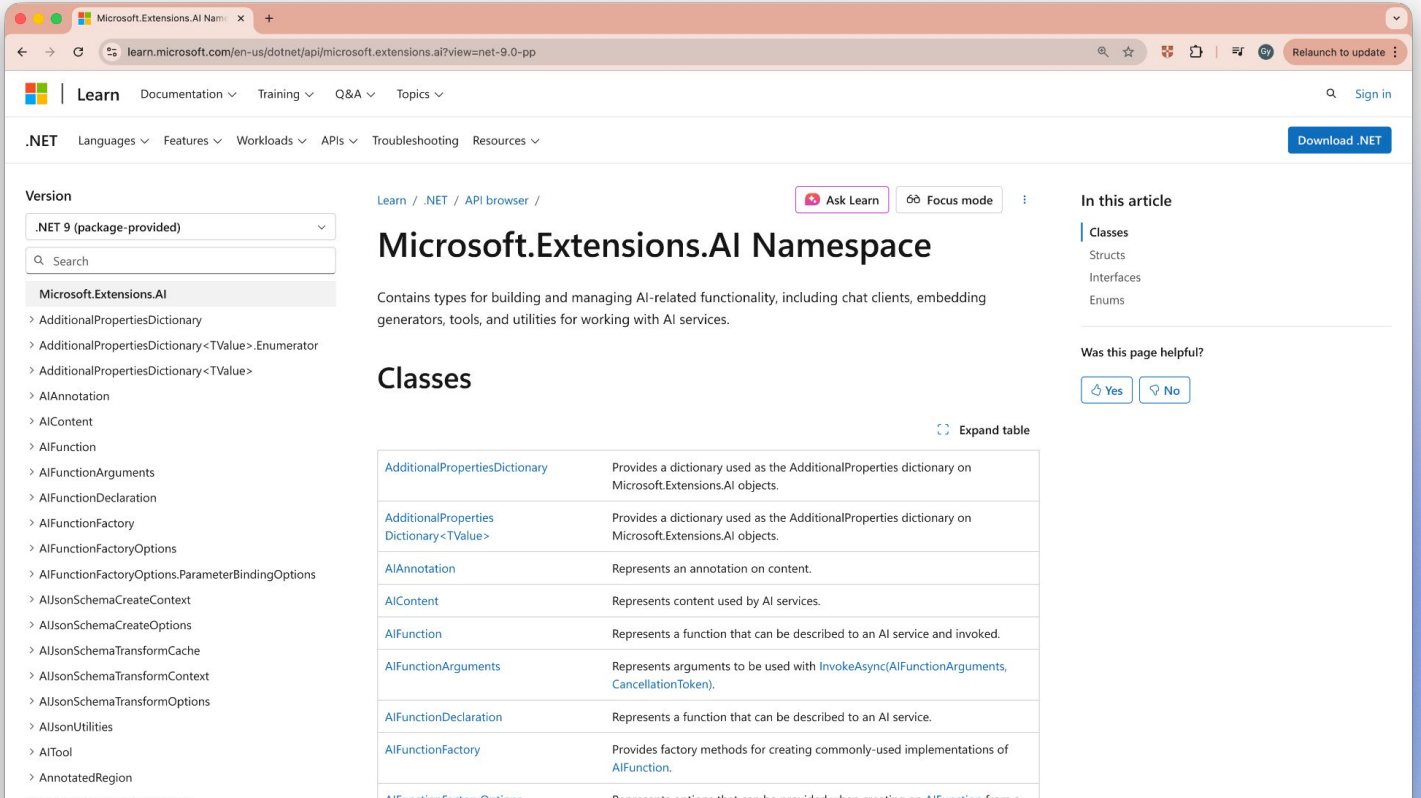


Microsoft.Extensions.AI

(MEAI)

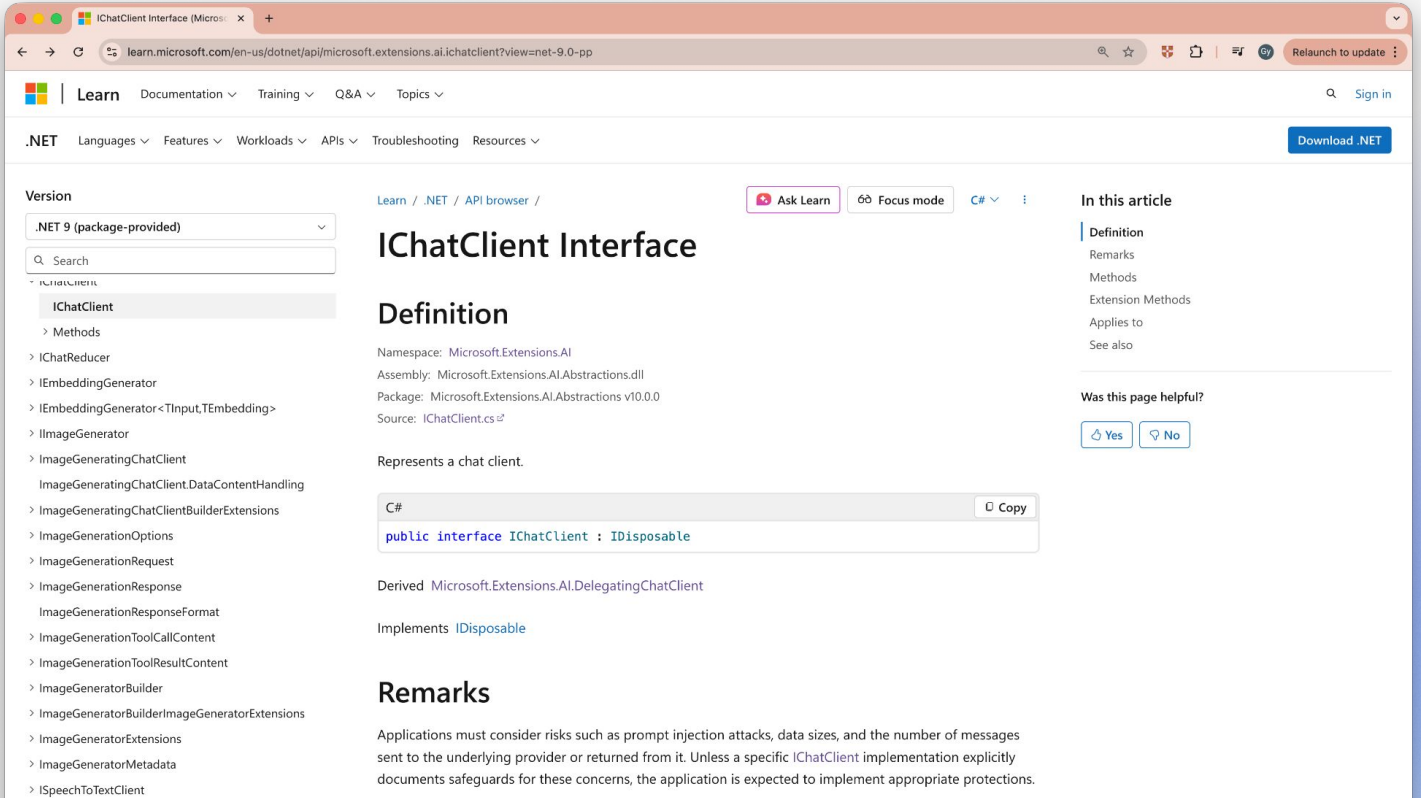
Microsoft.Extensions.AI

https://learn.microsoft.com/dotnet/api/microsoft.extensions.ai



Microsoft.Extensions.AI - IChatClient

https://learn.microsoft.com/dotnet/api/microsoft.extensions.ai



https://github.com/dotnet/extensions/blob/main/src/Libraries/Microsoft.Extensions.AI.Abstractions/ChatCompletion/IChatClient.cs

IChatClient.cs

```
namespace Microsoft.Extensions.AI;

public interface IChatClient : IDisposable
{
    Task<ChatResponse> GetResponseAsync(
        IEnumerable<ChatMessage> messages,
        ...);

    IAsyncEnumerable<ChatResponseUpdate> GetStreamingResponseAsync(
        IEnumerable<ChatMessage> messages,
        ...);

    object? GetService(Type serviceType, object? serviceKey = null);
}
```

IChatClient.cs

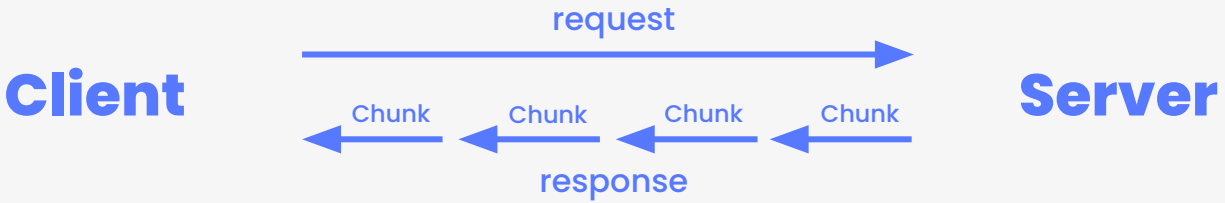
```
namespace Microsoft.Extensions.AI;

public interface IChatClient : IDisposable
{
    Task<ChatResponse> GetResponseAsync(
        IEnumerable<ChatMessage> messages,
        ...);

    IAsyncEnumerable<ChatResponseUpdate> GetStreamingResponseAsync(
        IEnumerable<ChatMessage> messages,
        ...);

    object? GetService(Type serviceType, object? serviceKey = null);
}
```

IChatClient.cs

```
namespace Microsoft.Extensions.AI;

public interface IChatClient : IDisposable
{
    Task<ChatResponse> GetResponseAsync(
        IEnumerable<ChatMessage> messages,
        ...);

    IAsyncEnumerable<ChatResponseUpdate> GetStreamingResponseAsync(
        IEnumerable<ChatMessage> messages,
        ...);

    object? GetService(Type serviceType, object? serviceKey = null);
}
```

```
Task<ChatResponse> GetStreamingResponse(...)
```


```
IAsyncEnumerable<ChatResponseUpdate> GetStreamingResponseAsync(...)
```



LLM Output: Streaming vs Non-Streaming

Submit

🧠 Non-Streaming Output

⚡ Streaming Output

EX. IChatClient::GetResponseAsync

```
using Microsoft.Extensions.AI;
using OllamaSharp;

IChatClient chatClient = new OllamaApiClient(
  new Uri("http://localhost:11434/"), "phi3:mini");

var result = await chatClient.GetResponseAsync("What is AI?");
Console.WriteLine(result);
```

EX. IChatClient::GetResponseAsync

```
using Microsoft.Extensions.AI;
using OllamaSharp;

IChatClient chatClient = new OllamaApiClient(
  new Uri("http://localhost:11434/"), "phi3:mini");

var result = await chatClient.GetResponseAsync("What is AI?");
Console.WriteLine(result);
```

EX. IChatClient::GetStreamingResponseAsync

```
using Microsoft.Extensions.AI;
using OllamaSharp;

IChatClient chatClient = new OllamaApiClient(
  new Uri("http://localhost:11434/"), "phi3:mini");

await foreach (ChatResponseUpdate item in chatClient.GetStreamingResponseAsync("What is AI?"))
{
        Console.Write(item.Text);
}
```

EX. IChatClient::GetStreamingResponseAsync

```
using Microsoft.Extensions.AI;
using OllamaSharp;

IChatClient chatClient = new OllamaApiClient(
  new Uri("http://localhost:11434/"), "phi3:mini");

await foreach (ChatResponseUpdate item in chatClient.GetStreamingResponseAsync("What is AI?"))
{
        Console.Write(item.Text);
}
```

💡 OpenChat Playground 에서는 MEAI를 어떻게 활용했을까?

OCP에서는 어떻게 활용했을까? - Key Idea 💡

Key Idea Pseudo Code

```
using Microsoft.Extensions.AI;

IChatClient chatClient = ConnectorFactory.Create(ConnectorType);
```

OCP에서는 어떻게 활용했을까? - Overview

src/OpenChat.PlaygroundApp/Program.cs

```
var builder = WebApplication.CreateBuilder(args);
...

var settings = ArgumentOptions.Parse(builder.Configuration, args);
...

var chatClient = await LanguageModelConnector.CreateChatClientAsync(settings);
...

builder.Services.AddChatClient(chatClient)
                .UseFunctionInvocation()
                ...;
...
builder.Services.AddScoped<IChatService, ChatService>();
...
```

OCP에서는 어떻게 활용했을까? - Overview

```
var builder = WebApplication.CreateBuilder(args);
```

⬆️ ASP.NET Core 애플리케이션 빌더 생성

```
var settings = ArgumentOptions.Parse(builder.Configuration, args);
...

var chatClient = await LanguageModelConnector.CreateChatClientAsync(settings);
...

builder.Services.AddChatClient(chatClient)
                .UseFunctionInvocation()
                ...;
...
builder.Services.AddScoped<IChatService, ChatService>();
...
```

OCP에서는 어떻게 활용했을까? - Overview

```
var builder = WebApplication.CreateBuilder(args);
...

var settings = ArgumentOptions.Parse(builder.Configuration, args);
⬆️ 설정 파싱하여 AppSettings 객체 생성
var chatClient = await LanguageModelConnector.CreateChatClientAsync(settings);
...

builder.Services.AddChatClient(chatClient)
                .UseFunctionInvocation()
                ...;
...
builder.Services.AddScoped<IChatService, ChatService>();
...
```

OCP에서는 어떻게 활용했을까? - Overview

```
var builder = WebApplication.CreateBuilder(args);
...

var settings = ArgumentOptions.Parse(builder.Configuration, args);
...

var chatClient = await LanguageModelConnector.CreateChatClientAsync(settings);
...
builder.Services.AddSingleton(chatClient)
                .UseFunctionInvocation()
                ...;

...
builder.Services.AddScoped<IChatService, ChatService>();
...
```

⬆️ 파싱된 설정에 따라 적절한 AI 모델 커넥터를 선택하고,
IChatClient 인스턴스 생성

OCP에서는 어떻게 활용했을까? - Overview

```
var builder = WebApplication.CreateBuilder(args);
...

var settings = ArgumentOptions.Parse(builder.Configuration, args);
...

    생성된 chatClient를 DI 컨테이너에
⬇️  채팅 클라이언트 서비스 (싱글톤) 등록

builder.Services.AddChatClient(chatClient)
                .UseFunctionInvocation()
                ...;

...
builder.Services.AddScoped<IChatService, ChatService>();
...
```

OCP에서는 어떻게 활용했을까? - Overview

```
var builder = WebApplication.CreateBuilder(args);
...

var settings = ArgumentOptions.Parse(builder.Configuration, args);
...

var chatClient = await LanguageModelConnector.CreateChatClientAsync(settings);
...

builder.Services.AddChatClient(chatClient)
...
```

IChatService 인터페이스와 ChatService 구현체를
⬇️ 스코프드 서비스로 등록하여 채팅 로직 캡슐화

```
builder.Services.AddScoped<IChatService, ChatService>();
...
```

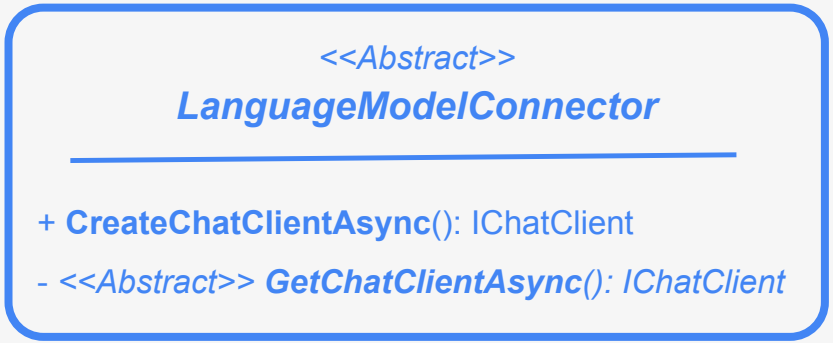
OCP에서는 어떻게 활용했을까? - Overview

```
var builder = WebApplication.CreateBuilder(args);
...

var settings = ArgumentOptions.Parse(builder.Configuration, args);
...

var chatClient = await LanguageModelConnector.CreateChatClientAsync(settings);
...

builder.Services.AddChatClient(chatClient)
                .UseFunctionInvocation()
                ...;
...
builder.Services.AddScoped<IChatService, ChatService>();
...
```

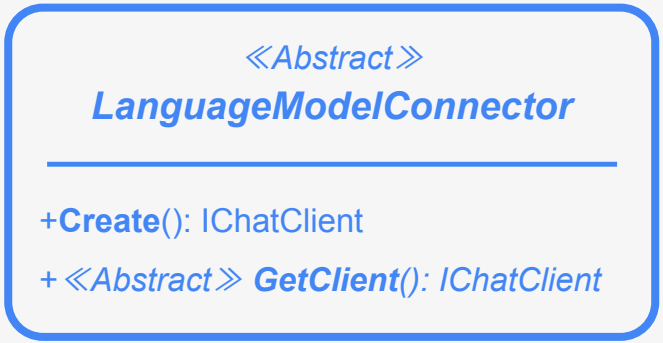
OCP에서는 어떻게 활용했을까? - Overview

```
var builder = WebApplication.CreateBuilder(args);
...

var settings = ArgumentOptions.Parse(builder.Configuration, args);
...

var chatClient = await LanguageModelConnector.CreateChatClientAsync(settings);
...

builder.Services.AddChatClient(chatClient)
                .UseFunctionInvocation()
                ...;
...
builder.Services.AddScoped<IChatService, ChatService>();
...
```

🏭 Factory Template Pattern: *LanguageModelConnector*

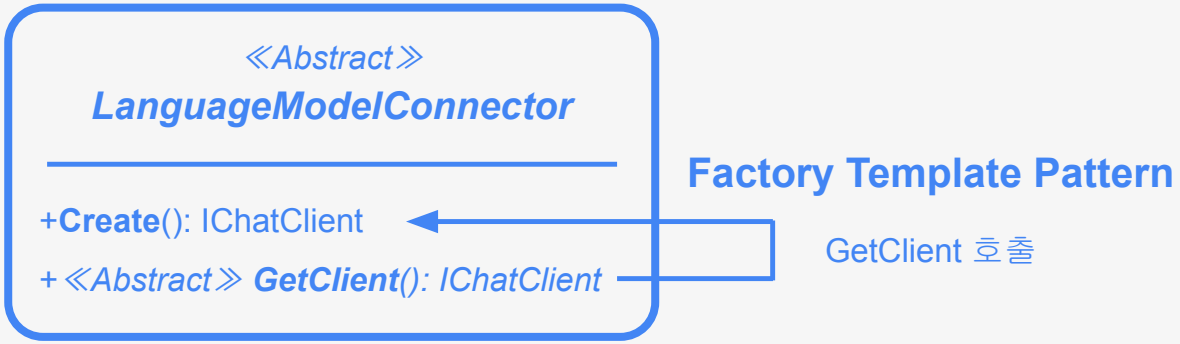


🏭 Factory Template Pattern: *LanguageModelConnector*



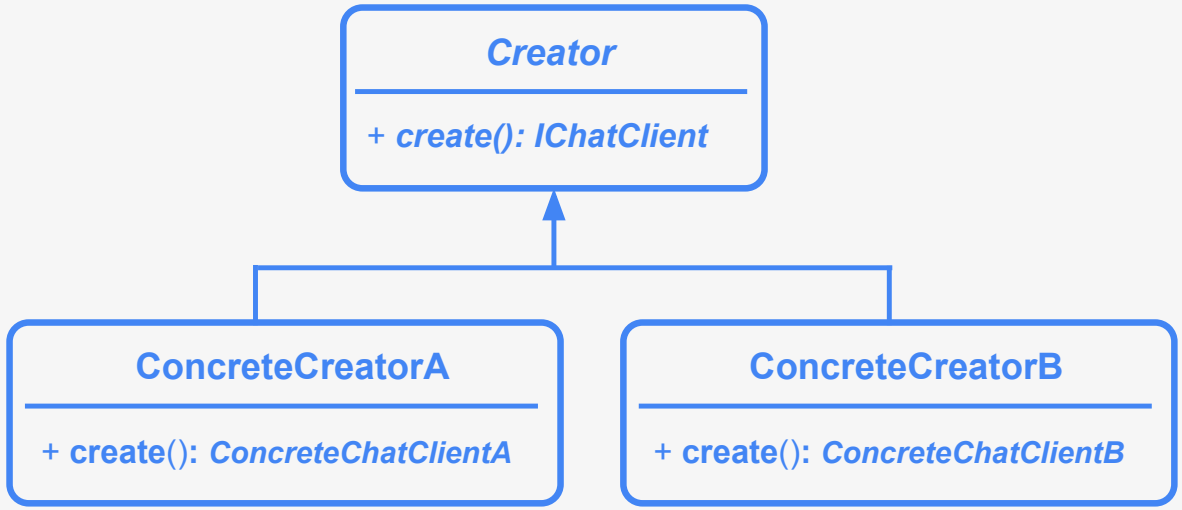
🏭 Factory Template Pattern: *LanguageModelConnector*

🏭 Factory Template Pattern: *LanguageModelConnector*



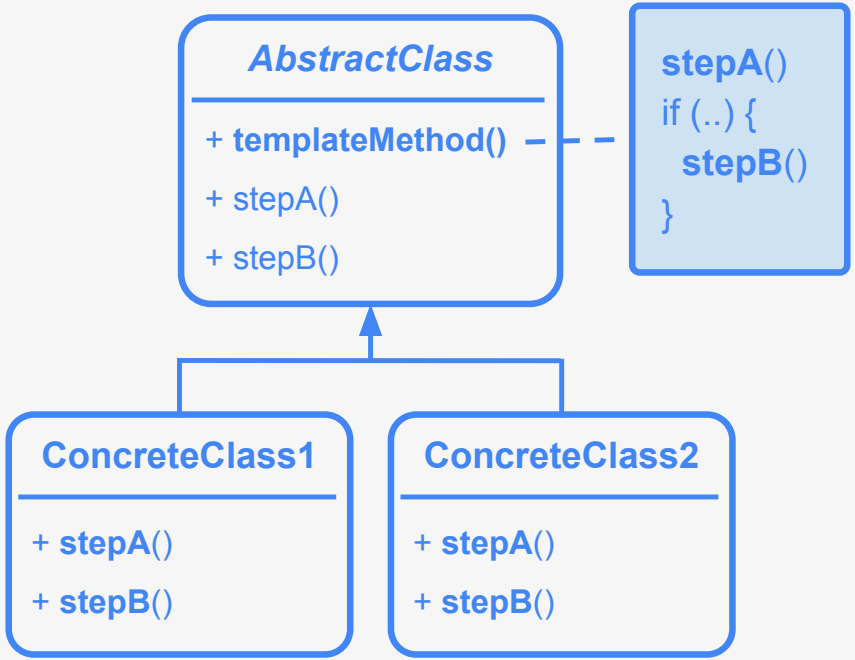
🏭 Factory Template Pattern: *LanguageModelConnector*

Factory Method Pattern

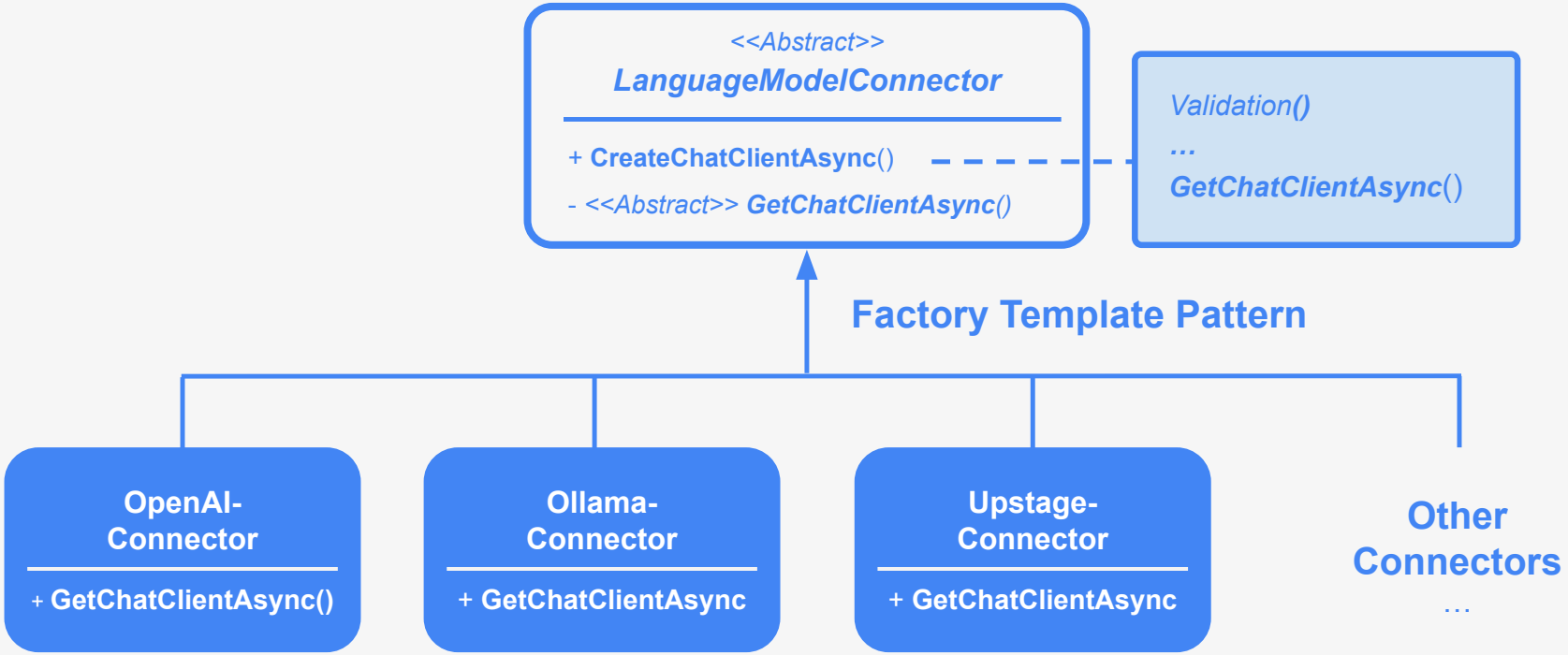


🏭 Factory Template Pattern: *LanguageModelConnector*

Template Method Pattern



🏭 Factory Template Pattern: *LanguageModelConnector*



🏭 Factory Template Pattern: *LanguageModelConnector*

```
public abstract class LanguageModelConnector(LanguageModelSettings? settings)
{
    // 모든 구현체가 IChatClient 구현체를 반환해야 함
    public abstract Task<IChatClient> GetChatClientAsync();

    // 🏭 Factory Method: 설정에 맞는 Connector로 IChatClient를 생성해서 반환
    public static async Task<IChatClient> CreateChatClientAsync(AppSettings settings)
    {
        LanguageModelConnector connector = settings.ConnectorType switch
        {
            ConnectorType.AmazonBedrock => new AmazonBedrockConnector(settings),
            ConnectorType.AzureAIFoundry => new AzureAIFoundryConnector(settings),
            ConnectorType.GitHubModels => new GitHubModelsConnector(settings),
            ...
            _ => throw new NotSupportedException($"Connector type '{settings.ConnectorType}' is not supported.")
        };

        connector.EnsureLanguageModelSettingsValid();

        return await connector.GetChatClientAsync().ConfigureAwait(false);
    }
}
```

🏭 Factory Template Pattern: *LanguageModelConnector*

```
public abstract class LanguageModelConnector(LanguageModelSettings? settings)
{
    // 모든 구현체가 IChatClient 구현체를 반환해야 함
    public abstract Task<IChatClient> GetChatClientAsync();

    // 🏭 Factory Method: 설정에 맞는 Connector로 IChatClient를 생성해서 반환
    public static async Task<IChatClient> CreateChatClientAsync(AppSettings settings)
    {
        LanguageModelConnector connector = settings.ConnectorType switch
        {
            ConnectorType.AmazonBedrock => new AmazonBedrockConnector(settings),
            ConnectorType.AzureAIFoundry => new AzureAIFoundryConnector(settings),
            ConnectorType.GitHubModels => new GitHubModelsConnector(settings),
            ...
            _ => throw new NotSupportedException($"Connector type '{settings.ConnectorType}' is not supported.")
        };

        connector.EnsureLanguageModelSettingsValid();

        return await connector.GetChatClientAsync().ConfigureAwait(false);
    }
}
```

🏭 Factory Template Pattern: *LanguageModelConnector*

```
public abstract class LanguageModelConnector(LanguageModelSettings? settings)
{
    // 모든 구현체가 IChatClient 구현체를 반환해야 함
    public abstract Task<IChatClient> GetChatClientAsync();

    // 🏭 Factory Method: 설정에 맞는 Connector로 IChatClient를 생성해서 반환
    public static async Task<IChatClient> CreateChatClientAsync(AppSettings settings)
    {
        LanguageModelConnector connector = settings.ConnectorType switch
        {
            ConnectorType.AmazonBedrock => new AmazonBedrockConnector(settings),
            ConnectorType.AzureAIFoundry => new AzureAIFoundryConnector(settings),
            ConnectorType.GitHubModels => new GitHubModelsConnector(settings),
            ...
            _ => throw new NotSupportedException($"Connector type '{settings.ConnectorType}' is not supported.")
        };

        connector.EnsureLanguageModelSettingsValid();

        return await connector.GetChatClientAsync().ConfigureAwait(false);
    }
}
```

🏭 Factory Template Pattern: *LanguageModelConnector*

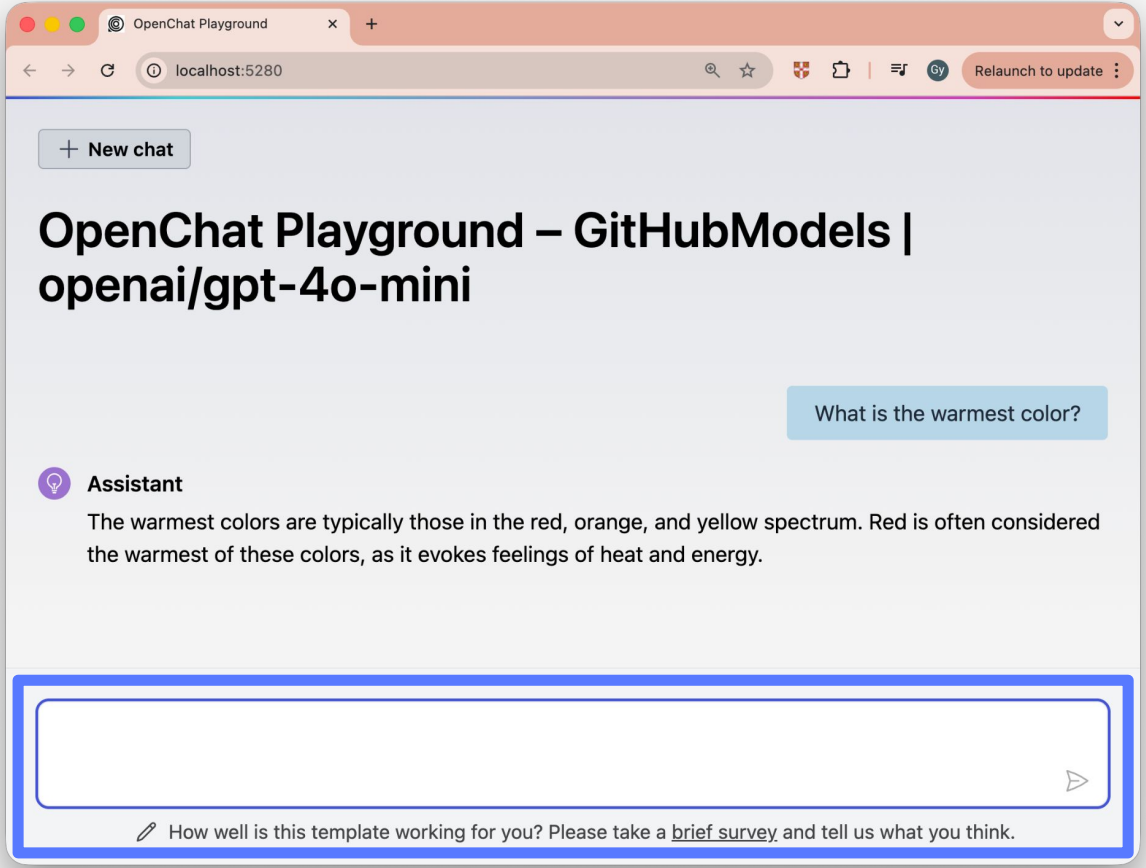
```
public abstract class LanguageModelConnector(LanguageModelSettings? settings)
{
    // 모든 구현체가 IChatClient 구현체를 반환해야 함
    public abstract Task<IChatClient> GetChatClientAsync();

    // 🏭 Factory Method: 설정에 맞는 Connector로 IChatClient를 생성해서 반환
    public static async Task<IChatClient> CreateChatClientAsync(AppSettings settings)
    {
        LanguageModelConnector connector = settings.ConnectorType switch
        {
            ConnectorType.AmazonBedrock => new AmazonBedrockConnector(settings),
            ConnectorType.AzureAIFoundry => new AzureAIFoundryConnector(settings),
            ConnectorType.GitHubModels => new GitHubModelsConnector(settings),
            ...
            _ => throw new NotSupportedException($"Connector type '{settings.ConnectorType}' is not supported.")
        };

        connector.EnsureLanguageModelSettingsValid();

        return await connector.GetChatClientAsync().ConfigureAwait(false);
    }
}
```

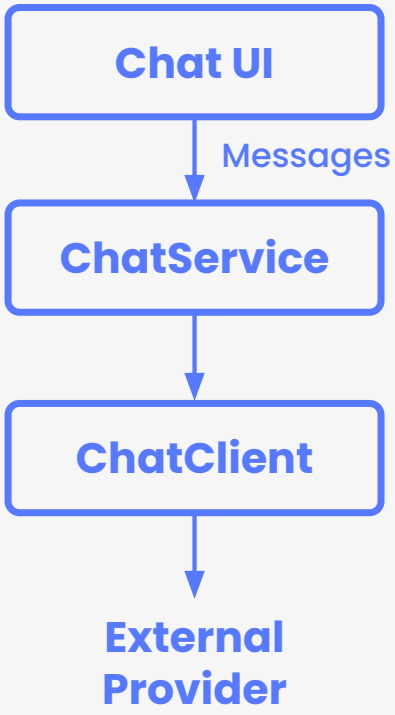
🏭 Factory Template Pattern: *LanguageModelConnector*

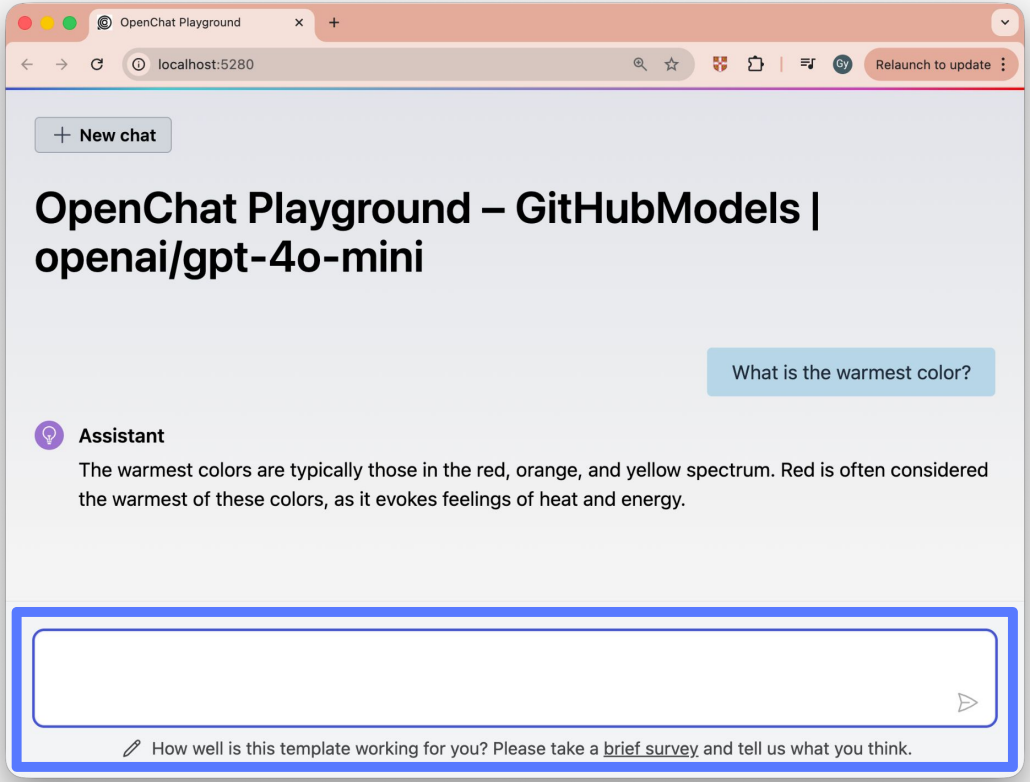
```
public abstract class LanguageModelConnector(LanguageModelSettings? settings)
{
    // 모든 구현체가 IChatClient 구현체를 반환해야 함
    public abstract Task<IChatClient> GetChatClientAsync();

    // 🏭 Factory Method: 설정에 맞는 Connector로 IChatClient를 생성해서 반환
    public static async Task<IChatClient> CreateChatClientAsync(AppSettings settings)
    {
        LanguageModelConnector connector = settings.ConnectorType switch
        {
            ConnectorType.AmazonBedrock => new AmazonBedrockConnector(settings),
            ConnectorType.AzureAIFoundry => new AzureAIFoundryConnector(settings),
            ConnectorType.GitHubModels => new GitHubModelsConnector(settings),
            ...
            _ => throw new NotSupportedException($"Connector type '{settings.ConnectorType}' is not supported.")
        };

        connector.EnsureLanguageModelSettingsValid();

        return await connector.GetChatClientAsync().ConfigureAwait(false);
    }
}
```

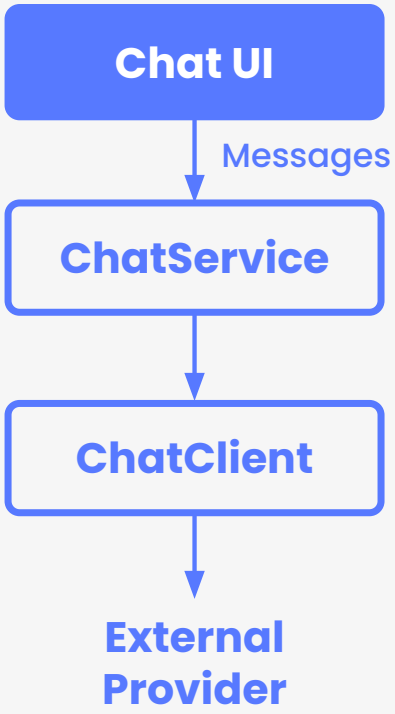
잘 만들어진 LLM Provider, 사용자의 요청에 어떻게 동작할까?

Flow: Web UI → External LLM Provider


OpenChat Playground
localhost:5280
Relaunch to update
New chat
OpenChat Playground – GitHubModels | openai/gpt-4o-mini
What is the warmest color?
Assistant
The warmest colors are typically those in the red, orange, and yellow spectrum. Red is often considered the warmest of these colors, as it evokes feelings of heat and energy.
How well is this template working for you? Please take a brief survey and tell us what you think.


Chat UI
Messages
ChatService
ChatClient
External Provider


OpenChat Playground
localhost:5280
Relaunch to update
New chat
OpenChat Playground – GitHubModels | openai/gpt-4o-mini
What is the warmest color?
Assistant
The warmest colors are typically those in the red, orange, and yellow spectrum. Red is often considered the warmest of these colors, as it evokes feelings of heat and energy.
How well is this template working for you? Please take a brief survey and tell us what you think.

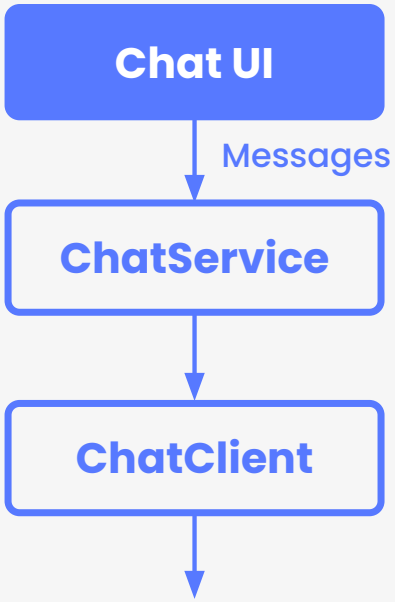


Chat.razor

```
<ChatInput OnSend="@AddUserMessageAsync" @ref="@chatInput" />
```

Chat.razor.cs

```
public partial class Chat : ComponentBase, IDisposable
{
    ...
    [Inject]
    public required IChatService ChatService { get; set; }
    ...

    private async Task AddUserMessageAsync(ChatMessage userMessage)
    {
        await foreach (var update in
            ChatService.GetStreamingResponseAsync(...)
        {
            messages.AddMessages(update, filter: c => c is not TextContent);
            ...
        }
    }
}
```

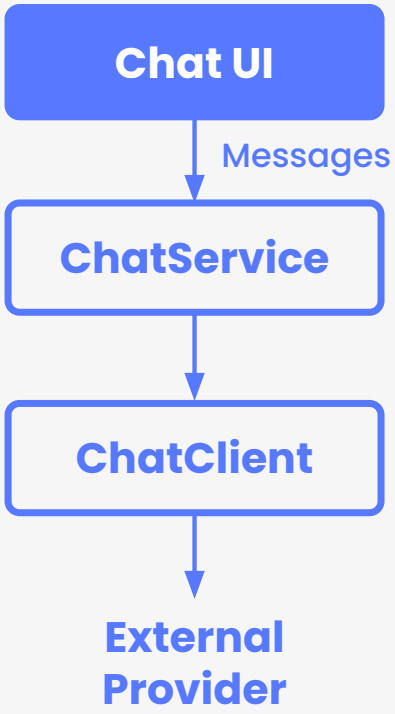


Chat.razor

```
<ChatInput OnSend="@AddUserMessageAsync" @ref="@chatInput" />
```

Chat.razor.cs

```
public partial class Chat : ComponentBase, IDisposable
{
    ...
    [Inject]
    public required IChatService ChatService { get; set; }
    ...

    private async Task AddUserMessageAsync(ChatMessage userMessage)
    {
        await foreach (var update in
            ChatService.GetStreamingResponseAsync(...)
        {
            messages.AddMessages(update, filter: c => c is not TextContent);
            ...
        }
    }
}
```



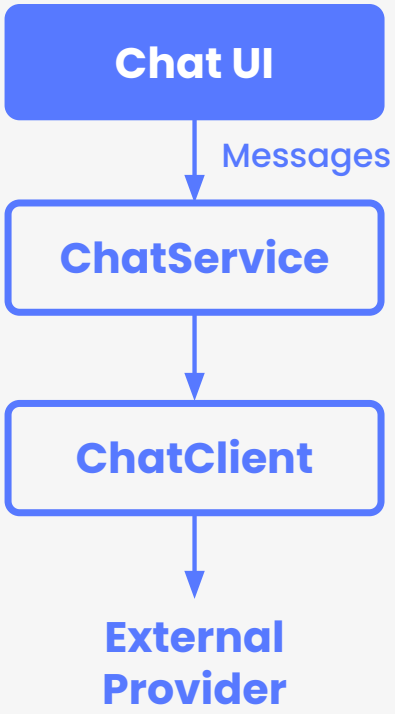
Chat.razor

```
<ChatInput OnSend="@AddUserMessageAsync" @ref="@chatInput" />
```

Chat.razor.cs

UI에서 ChatService를 의존성 주입 받아 사용

```
public partial class Chat : ComponentBase, IDisposable
{
    [Inject]
    public required IChatService ChatService { get; set; }
    ...

    private async Task AddUserMessageAsync(ChatMessage userMessage)
    {
        await foreach (var update in
            ChatService.GetStreamingResponseAsync(...)
        {
            messages.AddMessages(update, filter: c => c is not TextContent);
            ...
        }
    }
}
```



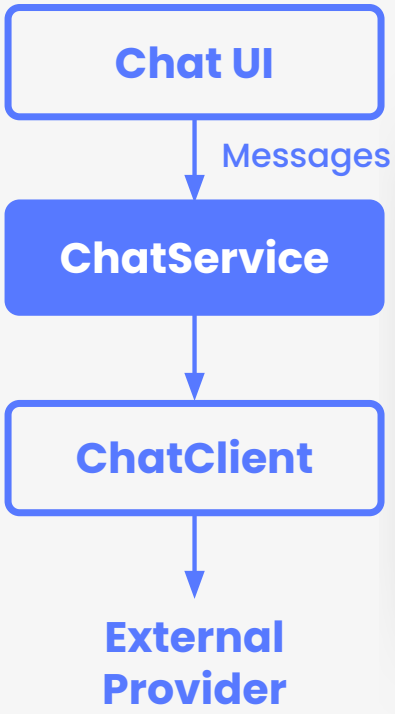
Chat.razor

```
<ChatInput OnSend="@AddUserMessageAsync" @ref="@chatInput" />
```

Chat.razor.cs

```
public partial class Chat : ComponentBase, IDisposable
{
    ...
    [Inject]
    public required IChatService ChatService { get; set; }
    ...

    private async Task AddUserMessageAsync(ChatMessage userMessage)
    {
        await foreach (var update in
            ChatService.GetStreamingResponseAsync(...)
        {
            messages.AddMessages(update, filter: c => c is not TextContent);
            ...
        }
    }
}
```

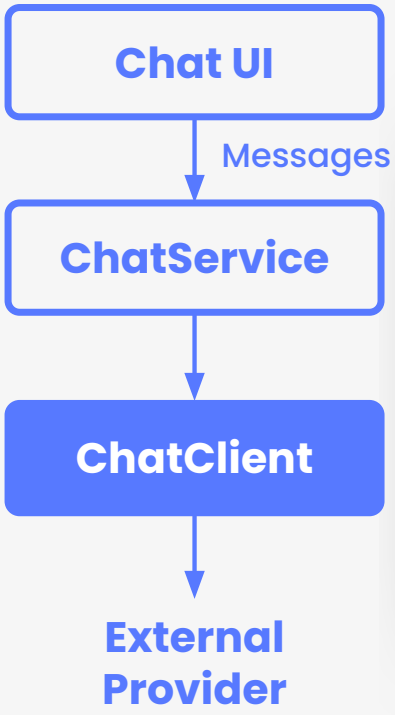
ChatService가 실제 유저 메시지 처리 전담



DI 컨테이너가 ChatService를 생성할 때
미리 생성된 ChatClient 인스턴스를 생성자 인자로 주입

ChatService.cs

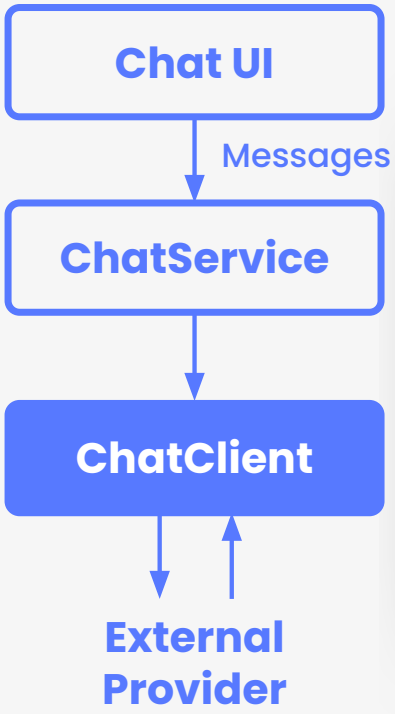
```
public class ChatService(IChatClient chatClient, ...) : IChatService
{
    private readonly IChatClient _chatClient = chatClient

    public IAsyncEnumerable<ChatResponseUpdate> GetStreamingResponseAsync(
        IEnumerable<ChatMessage> messages, ...
    {
        ...
        return this._chatClient.GetStreamingResponseAsync(messages, ...);
    }
}
```



각 LLM Provider 별 Connector 에서 생성한
IChatClient 구현체

ChatService.cs

```
public class ChatService IChatClient chatClient, ...) : IChatService
{
    private readonly IChatClient _chatClient = chatClient

    public IAsyncEnumerable<ChatResponseUpdate> GetStreamingResponseAsync(
        IEnumerable<ChatMessage> messages, ...
    {
        ...
        return this._chatClient.GetStreamingResponseAsync(messages, ...);
    }
}
```

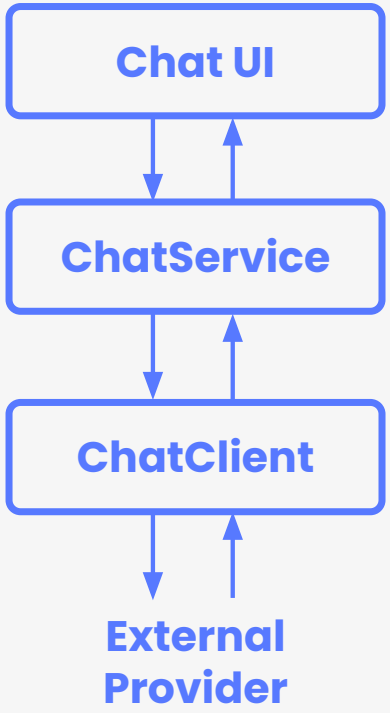


ChatService.cs

```
public class ChatService(IChatClient chatClient, ...) : IChatService
{
    private readonly IChatClient _chatClient = chatClient

    public IAsyncEnumerable<ChatResponseUpdate> GetStreamingResponseAsync(
        IEnumerable<ChatMessage> messages, ...
    {
        ...
        return this._chatClient.GetStreamingResponseAsync(messages, ...);
    }
}
```

LLM Provider Connector가 생성한
ChatClient 구현체를 사용해
스트리밍 Chat Completion 호출

📚 사용자의 요청부터 LLM Provider 까지



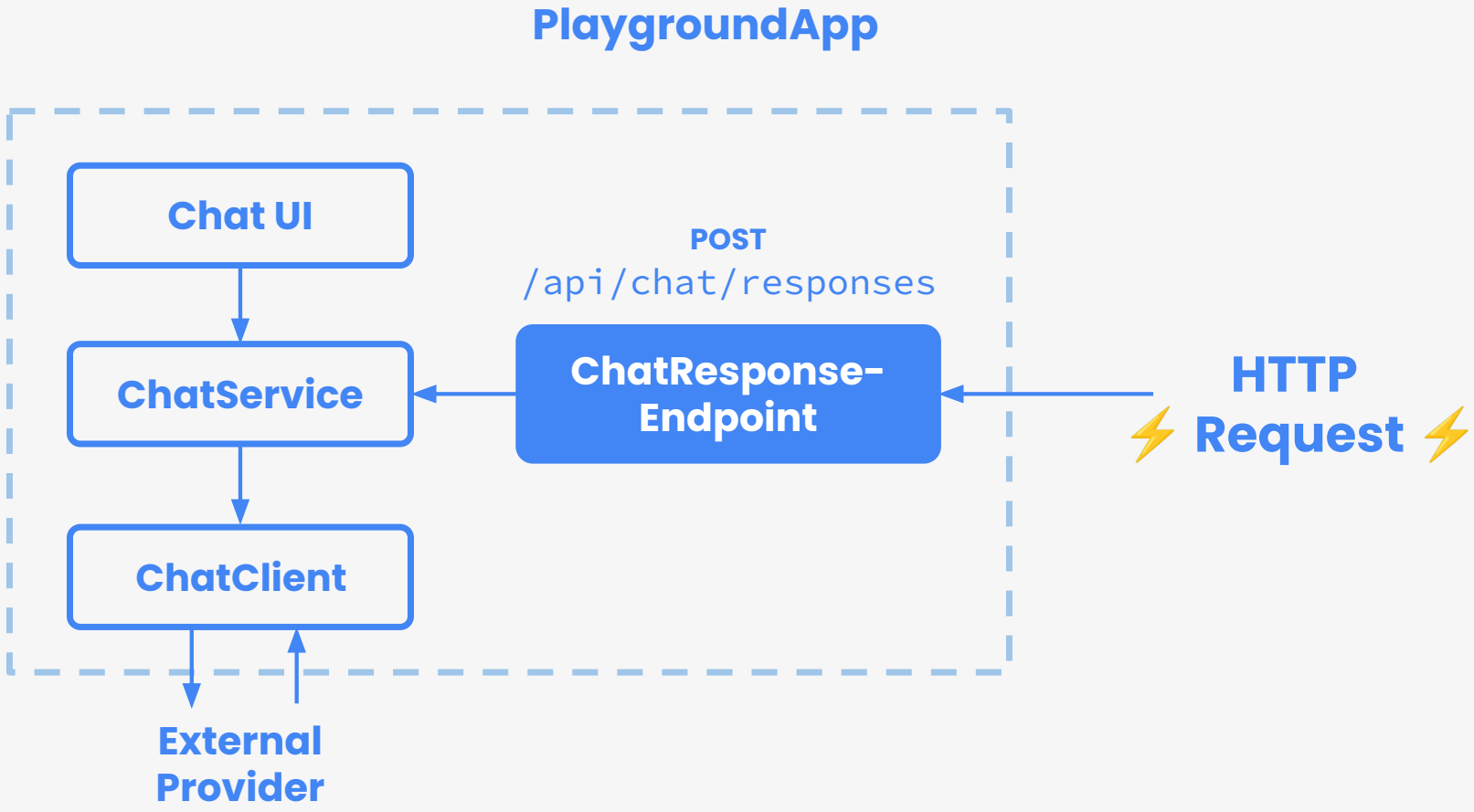
백엔드 API가 필요하다구요?

백엔드 API가 필요하다구요?

여기 있습니다 🤝

⬇️⬇️⬇️⬇️⬇️⬇️

/api/chat/responses


PlaygroundApp
Chat UI
ChatService
ChatClient
POST
/api/chat/responses
ChatResponse-
Endpoint
HTTP
⚡ Request ⚡
External
Provider

Demo

Chat.razor.cs

```
curl -N -X POST http://localhost:5280/api/chat/responses \
  -H "Content-Type: application/json" \
  -d '[
        {
          "role":"system",
          "message":"You are a helpful assistant."
        },
        {
          "role":"user",
          "message":"What is the warmest color?"
        }
      ]'
```

Code File Edit Selection View Go Run Terminal Window Help

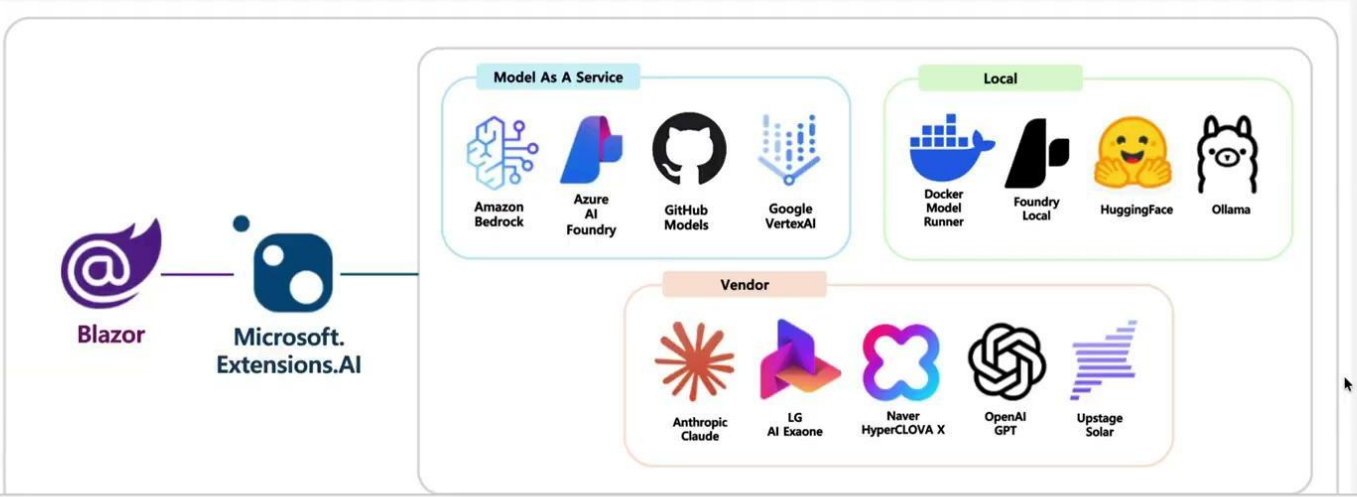
Mon Nov 24 12:35 PM

open-chat-playground

Preview README.md | curl-example.txt U | README.md

Open Chat Playground

Open Chat Playground (OCP) is a web UI that is able to connect virtually any LLM from any platform.

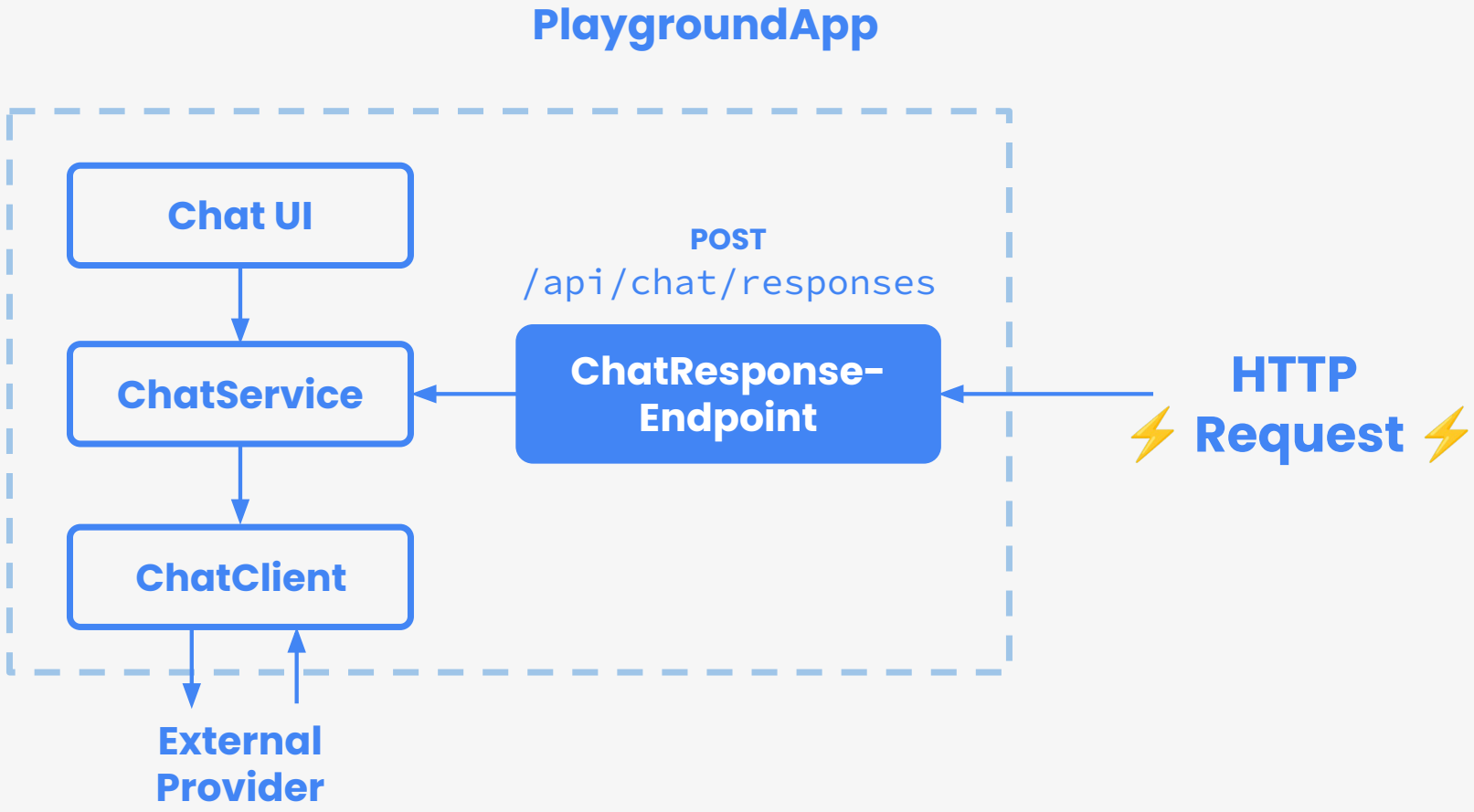


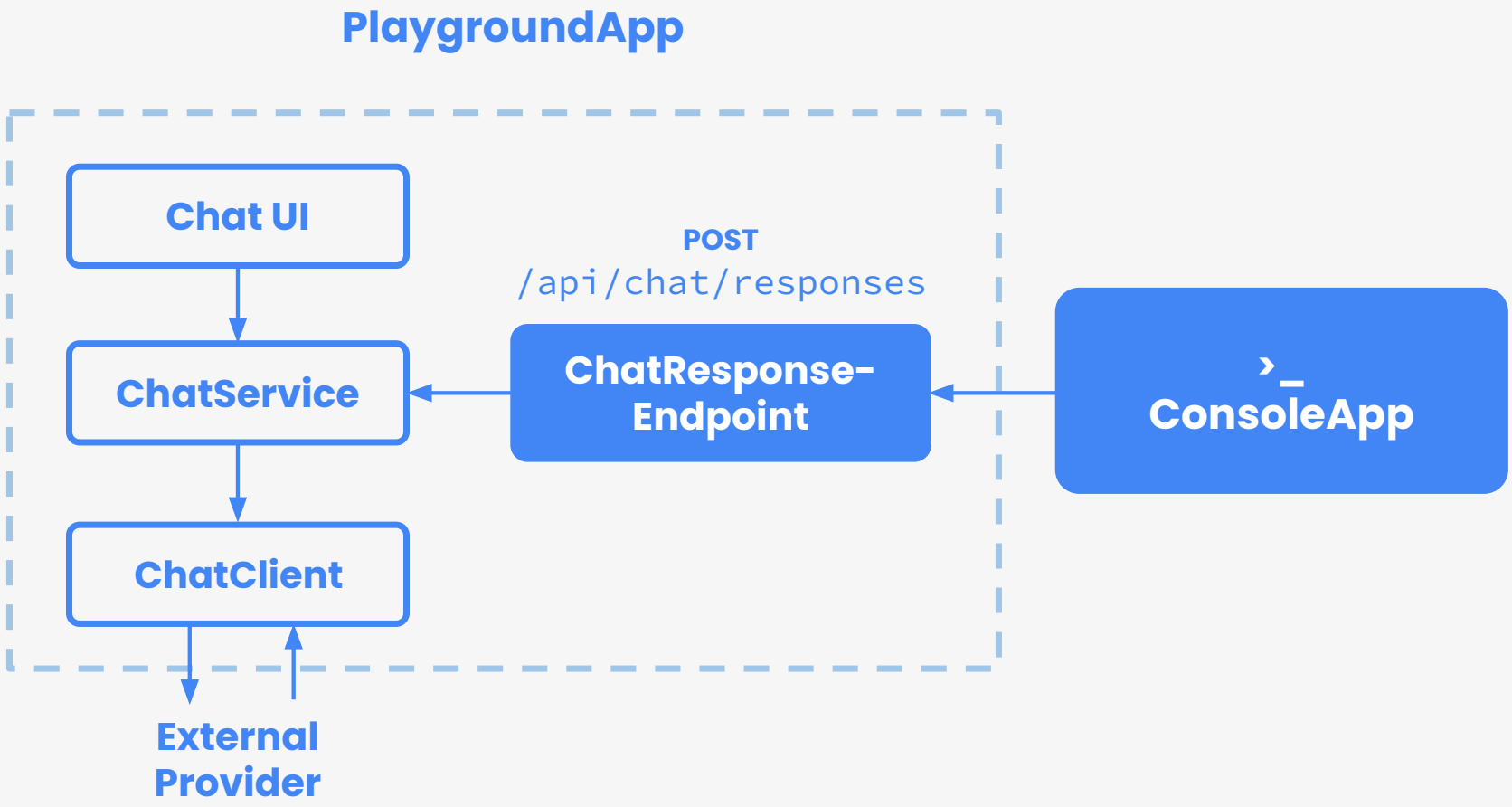
PROBLEMS 8 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

olla...

zsh...

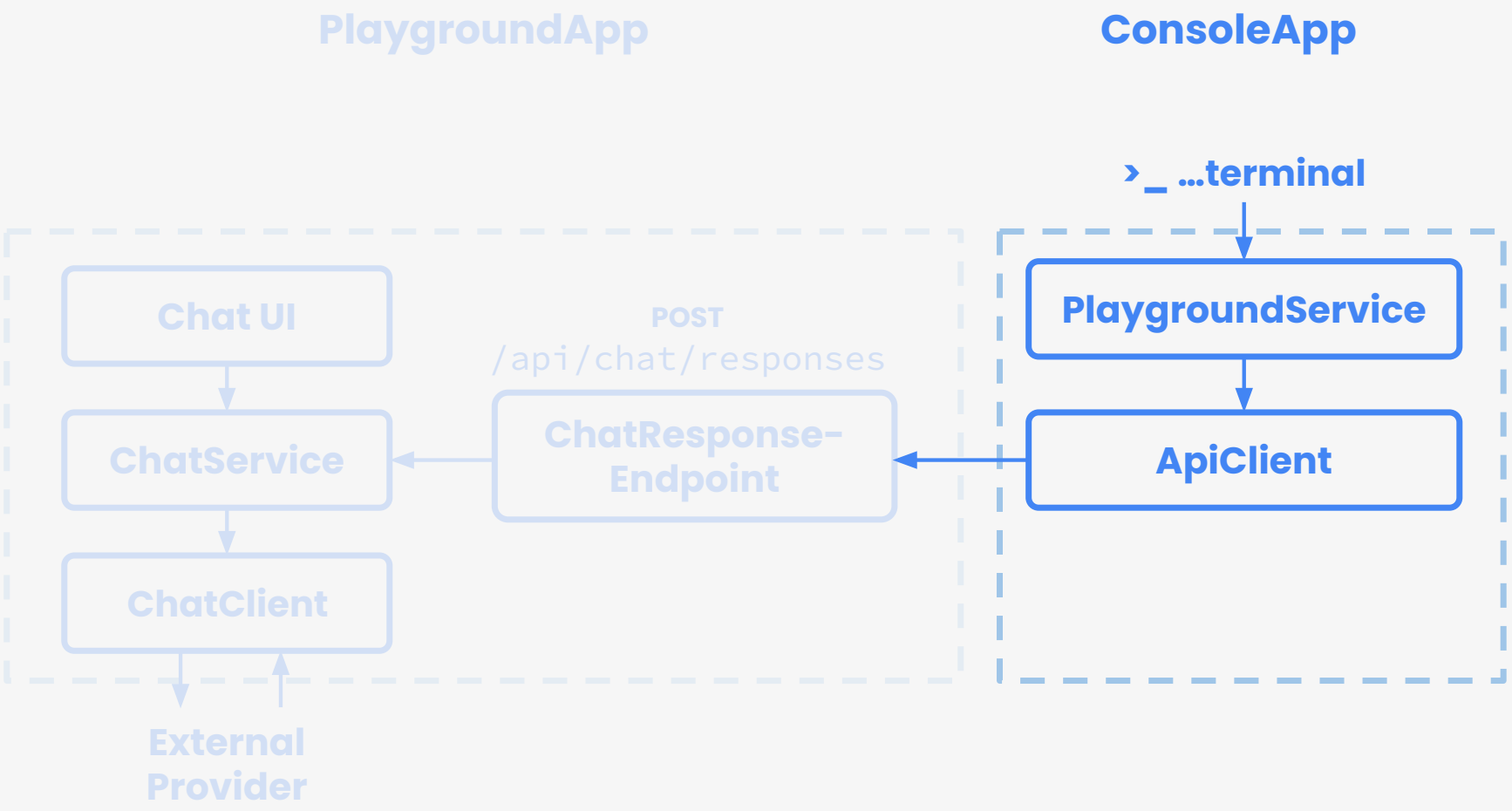
main* 0↓ 2↑ 0 8 Projects: 4 Debug Any CPU Solution file opened: OpenChatPlayground.sln Git Graph


PlaygroundApp
Chat UI
ChatService
ChatClient
POST
/api/chat/responses
ChatResponse-
Endpoint
HTTP
⚡ Request ⚡
External
Provider


PlaygroundApp
Chat UI
ChatService
ChatClient
External Provider
POST
/api/chat/responses
ChatResponse-Endpoint
>_
ConsoleApp

🏗️ Project Structure

```
📁open-chat-playground/
├── src/
│   ├── OpenChat.PlaygroundApp/          # 웹 애플리케이션
│   │   ├── Components/                  # Blazor 컴포넌트
│   │   ├── Abstractions/                # 추상화 레이어
│   │   ├── Connectors/                  # LLM 제공업체 커넥터
│   │   ├── Configurations/              # 설정 클래스
│   │   ├── Endpoints/                   # API 엔드포인트
│   │   └── Services/                    # 비즈니스 로직
│   └── OpenChat.ConsoleApp/             # CLI 애플리케이션
│       ├── Clients/                     # HTTP 클라이언트
│       ├── Services/                    # 서비스 로직
│       └── Options/                     # 설정 옵션
├── test/                                # 테스트
├── docs/                                # 문서
├── infra/                               # Azure 인프라
└── README.md
```


PlaygroundApp
ConsoleApp
›_ ...terminal
Chat UI
ChatService
ChatClient
POST
/api/chat/responses
ChatResponse-
Endpoint
PlaygroundService
ApiClient
External
Provider

Code File Edit Selection View Go Run Terminal Window Help

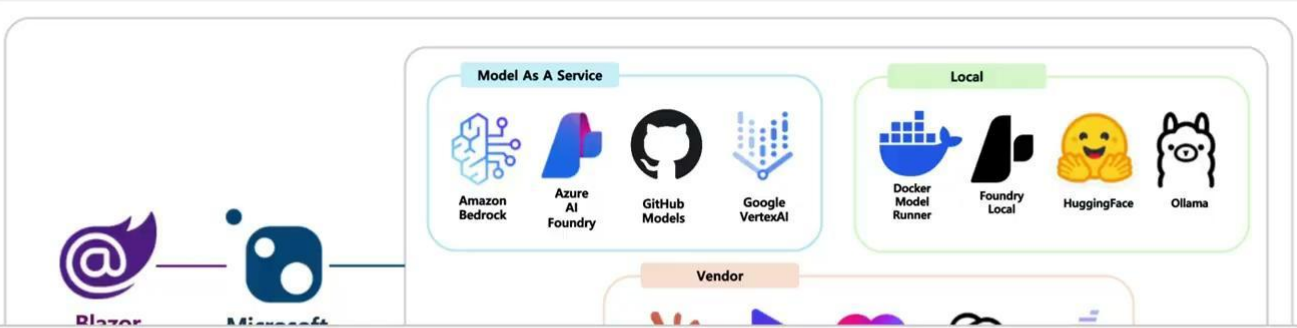
Mon Nov 24 12:40 PM

open-chat-playground

Preview README.md

Open Chat Playground

> Open Chat Playground (OCP) is a web UI that is able to connect virtually any LLM from any platform.



PROBLEMS 8 TERMINAL

zsh - OpenChat.PlaygroundApp

~/g/0/open-chat-playground/s/OpenChat.PlaygroundApp on main ⇡2 ?2

main* 0↓ 2↑ 0 8 Projects: 4 Debug Any CPU Solution file opened: OpenChatPlayground.sln Git

One API. Any Device. ⚡

Chat UI

ChatService

ChatClient

External Provider

POST

/api/chat/responses

ChatResponse-Endpoint

Mobile

Desktop

Web Browser

🚀 빌드 환경

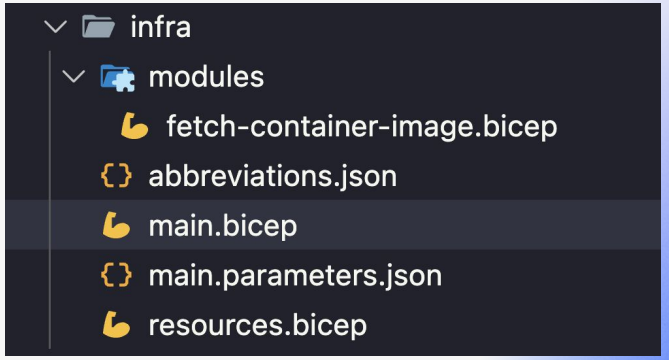
- ✅ **Local** 환경
- ✅ **Container** 환경
- ✅ **Azure Container App** 배포

🏗️ Project Structure

```
📁open-chat-playground/
├── src/
│   ├── OpenChat.PlaygroundApp/          # 웹 애플리케이션
│   └── OpenChat.ConsoleApp/             # CLI 애플리케이션
├── test/                                # 테스트
├── docs/                                # 문서
├── infra/                               # Azure 인프라
└── README.md
```

🏗️ Project Structure

```
📁open-chat-playground/
├── src/
│   ├── OpenChat.PlaygroundApp/
│   └── OpenChat.ConsoleApp/
├── test/
├── docs/
├── infra/
└── README.md
```



🚀 빌드 환경

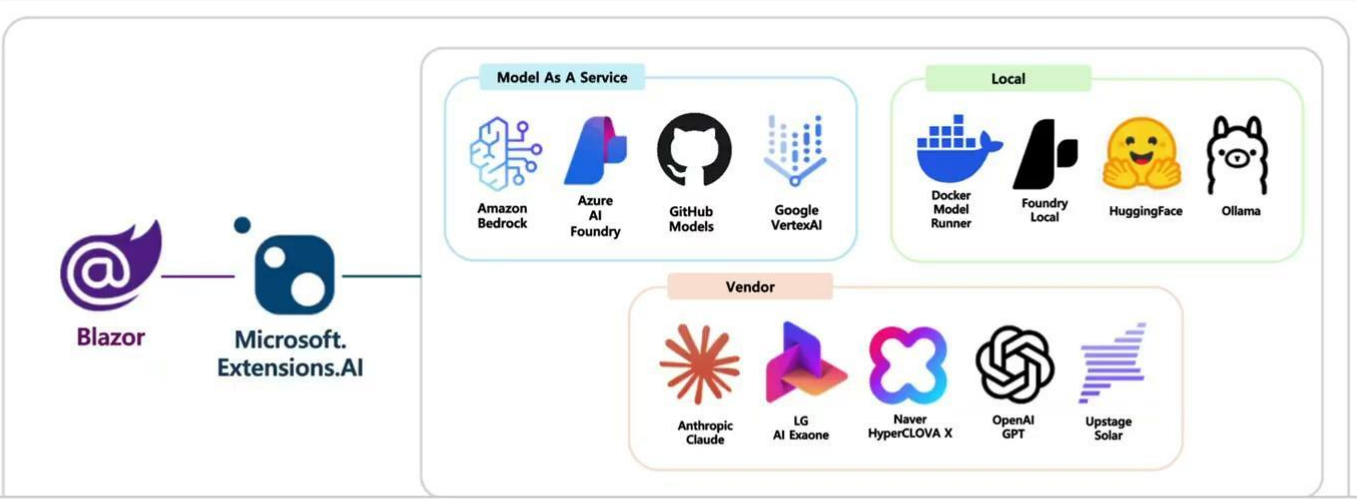
✅ Local 환경

✅ **Container 환경**

✅ Azure Container App 배포

Open Chat Playground

Open Chat Playground (OCP) is a web UI that is able to connect virtually any LLM from any platform.



🚀 빌드 환경

- ✅ **Local** 환경
- ✅ **Container** 환경
- ✅ **Azure Container App** 배포

🧭 Roadmap

🧭 Roadmap

- ✅ LLM 제공 지원 확대
- ☑️ Real-time LLM 교체
- ☑️ Aspire Orchestration
- ☑️ MCP 서버 지원

🧭 Roadmap

- ✅ LLM 제공 지원 확대
- ✅ Real-time LLM 교체
- ☑️ Aspire Orchestration
- ☑️ MCP 서버 지원

🧭 Roadmap

- ✅ LLM 제공 지원 확대
- ✅ Real-time LLM 교체
- ✅ Aspire Orchestration
- ☑️ MCP 서버 지원

🧭 Roadmap

- ✅ LLM 제공 지원 확대
- ✅ Real-time LLM 교체
- ✅ Aspire Orchestration
- ✅ MCP 서버 지원

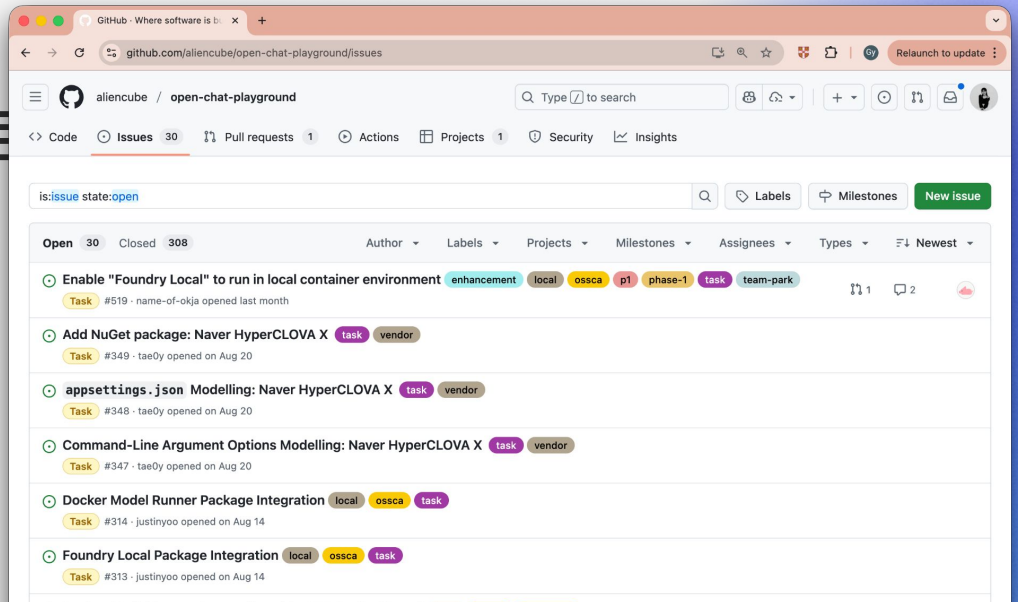
📖 Open Source

📖 Open Source

✅ 누구나 기여 가능

✅ 현행화된 문서화

✅ 지속적인 업데이트



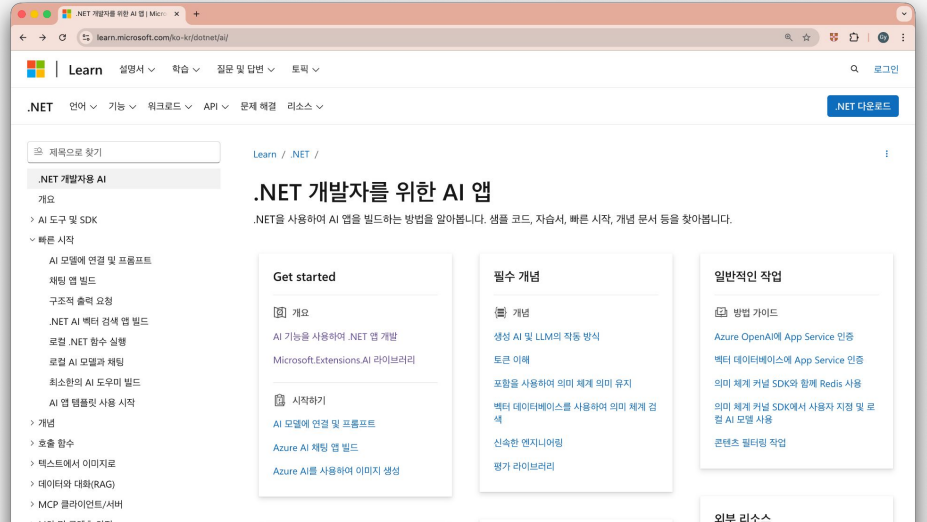
OpenChat Playground

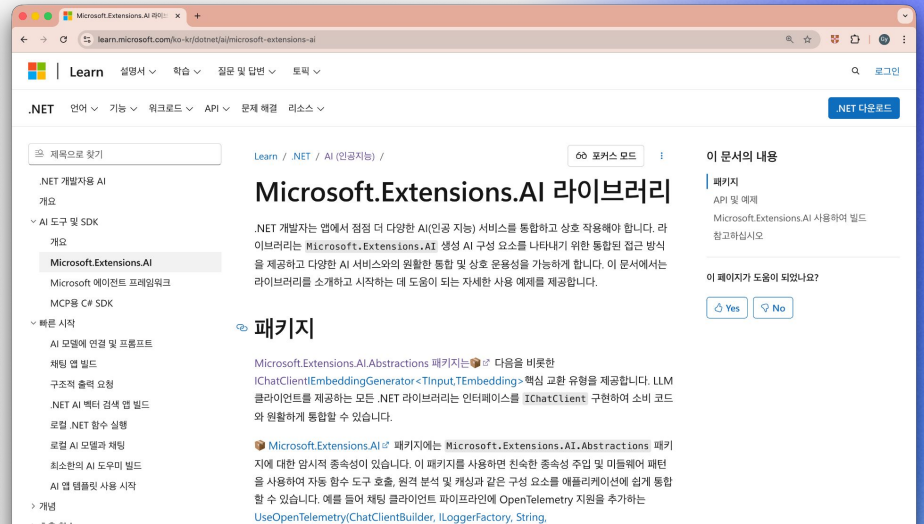
https://bit.ly/openchat-playground

📚 더 알아보고 싶다면

🔗 https://learn.microsoft.com/dotnet/ai

🔗 https://learn.microsoft.com/dotnet/ai/microsoft-extensions-ai





EOD